

Table of Contents

1	Introduction	2
2	System Analysis	9
3	System Design	32
4	Coding	46
5	Standardization of the Coding	118
6	Testing	122
7	System Security Mechanisms	126
8	Results	131
9	Future Enhancements	141
10	Conclusion	143
11	Bibliography	148

Introduction

1.1 Introduction

Healthcare plays a vital role in improving the quality of life and ensuring the well-being of society. However, many rural and underserved regions continue to face significant challenges in accessing timely healthcare services due to limited medical infrastructure, shortage of healthcare professionals, delayed disease reporting, and inefficient coordination among hospitals, field workers, and health authorities. During disease outbreaks and public health emergencies, these limitations often lead to delayed responses, ineffective resource allocation, and increased health risks for the population. Traditional healthcare management systems primarily depend on manual reporting and fragmented communication, making it difficult for authorities to monitor disease trends, organize vaccination campaigns, and deploy healthcare resources efficiently.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting project is proposed as an intelligent healthcare management platform that leverages Artificial Intelligence (AI), Machine Learning (ML), Geographic Information Systems (GIS), and real-time healthcare data to enhance public health decision-making. The system integrates information collected from hospitals, field workers, and healthcare administrators into a centralized platform capable of monitoring disease reports, forecasting disease trends, mapping healthcare accessibility, and generating health alerts for high-risk regions. By analyzing historical disease records, healthcare resource availability, regional demographics, and environmental factors, the system provides predictive insights that help authorities identify potential disease outbreaks before they become widespread.

The project also introduces an efficient coordination mechanism between Administrators, Hospitals, Field Workers, and Users. Hospitals can register and update disease reports as well as healthcare resources such as ICU beds, ambulances, and total bed availability. Field workers can update vaccination campaign progress, conduct regional health surveys, monitor assigned healthcare units, and report vaccination

completion details from the field. Administrators can manage hospitals, healthcare regions, diseases, mobile healthcare units, health alerts, vaccination campaigns, and system users while approving or rejecting hospital registrations. Through this collaborative workflow, healthcare information is collected in real time and shared among all stakeholders, enabling faster response during disease outbreaks and improving healthcare service delivery.

The primary objective of the proposed system is to transform conventional reactive healthcare management into a proactive, intelligent, and data-driven healthcare ecosystem. AI-based disease forecasting assists health authorities in predicting future disease trends and identifying high-risk regions, while GIS-based healthcare access mapping highlights underserved areas requiring immediate medical attention. The system further supports healthcare planning by optimizing the deployment of mobile healthcare units, organizing vaccination campaigns, issuing public health alerts, and improving resource utilization across hospitals and healthcare facilities.

The project scope includes the development of a secure web-based healthcare management platform featuring user authentication, role-based access control, healthcare facility management, disease reporting, healthcare access mapping, AI-based disease prediction, vaccination campaign management, regional survey management, health alert generation, hospital resource monitoring, and centralized administrative dashboards. The platform is designed to ensure secure handling of healthcare information while providing real-time analytics, visualization dashboards, and predictive reports for healthcare administrators and policymakers. The system does not replace clinical diagnosis or direct patient treatment; instead, it functions as an intelligent decision-support system that assists healthcare organizations in planning, monitoring, and responding to public health challenges more effectively.

The target users of the system include Healthcare Administrators, Government Health Departments, Hospitals, Field Workers, Public Health Officials, and Citizens. Administrators utilize the platform to manage healthcare infrastructure and monitor overall system performance. Hospitals contribute disease reports and update healthcare resource availability. Field workers perform on-site healthcare activities such as vaccination reporting, regional surveys, and healthcare monitoring. Public health authorities use AI-generated predictions and healthcare accessibility maps to plan healthcare interventions and allocate resources efficiently, while citizens benefit

indirectly through improved healthcare accessibility, faster disease outbreak response, and better public health services.

Overall, the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system provides an integrated, intelligent, and scalable solution for modern healthcare management by combining artificial intelligence, predictive analytics, geospatial mapping, and collaborative healthcare workflows. The platform supports evidence-based decision-making, enhances disease surveillance, improves healthcare accessibility, and strengthens public health preparedness, ultimately contributing to a more efficient, equitable, and resilient healthcare system.

1.2 Background of the Study

The rapid growth of population, increasing disease outbreaks, and unequal distribution of healthcare facilities have created significant challenges in providing timely and effective healthcare services. Many rural and remote regions continue to experience limited access to hospitals, medical professionals, and emergency healthcare services. During disease outbreaks, delays in reporting disease cases, lack of coordination between healthcare institutions, and inefficient allocation of medical resources often result in the rapid spread of diseases and increased pressure on healthcare systems. Traditional healthcare management methods mainly rely on manual reporting and disconnected information systems, making it difficult for health authorities to monitor disease trends and respond quickly to public health emergencies.

Existing healthcare management systems are largely fragmented and lack real-time monitoring, predictive analytics, and centralized coordination among hospitals, field workers, and government health departments. Hospitals often submit disease reports manually, while field workers maintain vaccination records and regional survey details separately, leading to delays in information sharing and decision-making. Furthermore, healthcare administrators face challenges in identifying high-risk regions, monitoring healthcare resource availability, organizing vaccination campaigns, and deploying mobile healthcare units effectively. The absence of an integrated platform with

intelligent forecasting capabilities limits the ability of authorities to take preventive actions before disease outbreaks become widespread.

Recent advancements in Artificial Intelligence (AI), Machine Learning (ML), Geographic Information Systems (GIS), and web-based technologies provide an effective solution to these challenges by enabling real-time healthcare monitoring and predictive healthcare management. The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system has been developed to integrate hospitals, field workers, healthcare administrators, and citizens through a centralized digital platform. The system allows hospitals to submit disease reports and update healthcare resource availability, enables field workers to conduct regional health surveys and manage vaccination campaigns, and provides administrators with tools to manage healthcare regions, diseases, health alerts, mobile healthcare units, and healthcare resources. By incorporating AI-based disease trend forecasting, healthcare accessibility mapping, health alerts, vaccination management, and centralized administrative monitoring, the proposed system aims to improve healthcare accessibility, enhance disease surveillance, optimize healthcare resource allocation, strengthen public health preparedness, and support faster, data-driven decision-making for improved healthcare service delivery.

1.3 Problem Definition

The current healthcare management system relies heavily on manual reporting and fragmented information, making it difficult to monitor disease outbreaks, healthcare resources, and vaccination activities in real time. Hospitals, field workers, and health authorities often work independently, leading to delayed communication, inefficient resource allocation, and slow response during public health emergencies. The absence of AI-based disease forecasting and healthcare access mapping further limits the ability to identify high-risk regions and provide timely medical interventions.

The formal Problem Statement is:

"The inability to integrate and analyze real-time healthcare data from hospitals, field workers, and healthcare authorities through a centralized intelligent platform leads to delayed disease detection, inefficient healthcare resource management, poor healthcare accessibility, and ineffective public health decision-making."

The core challenge is to develop an AI-powered healthcare management system that integrates disease reporting, healthcare access mapping, vaccination management, health alerts, and predictive analytics into a single platform, enabling proactive decision-making, efficient resource utilization, and improved healthcare service delivery.

1.4 Objectives of the Project

The primary objective of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system is to improve public healthcare management by providing real-time disease monitoring, healthcare accessibility analysis, and AI-based disease trend forecasting. The system aims to support healthcare authorities in making faster, data-driven decisions, optimizing healthcare resources, and improving healthcare services through an integrated digital platform.

The specific objectives of the system include:

1. To develop an AI-based disease trend forecasting system that predicts disease outbreaks using historical and real-time healthcare data.
2. To provide real-time healthcare access mapping for identifying underserved regions and healthcare accessibility gaps.
3. To integrate healthcare data from hospitals, field workers, healthcare facilities, and regional surveys into a centralized database.
4. To enable hospitals to submit disease reports and update healthcare resource availability, including ICU beds, ambulances, and total beds.
5. To assist field workers in managing vaccination campaigns, updating vaccination completion reports, and conducting regional health surveys.
6. To generate health alerts and early warning notifications for high-risk regions to support timely preventive actions.
7. To optimize the deployment of mobile healthcare units based on disease risk, healthcare accessibility, and regional healthcare needs.

8. To provide administrators with tools to manage hospitals, healthcare regions, diseases, health alerts, vaccination campaigns, and mobile healthcare units.
 9. To maintain a centralized database for storing disease reports, healthcare resources, vaccination records, survey details, and healthcare access information.
 10. To provide a secure and user-friendly web-based platform with role-based access for Administrators, Hospitals, Field Workers, and Users.
 11. To support healthcare authorities with real-time dashboards, analytical reports, and AI-generated insights for effective planning and decision-making.
 12. To ensure system security, data privacy, reliability, and scalability through authenticated access and secure data management using the Django framework.
 13. To improve public healthcare services by enabling proactive disease surveillance, efficient resource allocation, and faster response to healthcare emergencies.
-

1.5 Scope of the Project

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system provides an intelligent web-based platform that helps healthcare authorities monitor disease trends, improve healthcare accessibility, and optimize healthcare resource management using Artificial Intelligence (AI), Machine Learning (ML), and Geographic Information Systems (GIS). The system focuses on disease forecasting, healthcare access mapping, vaccination management, and real-time decision support.

The scope of the project includes:

- **Disease Trend Forecasting:** Predicting disease outbreaks by analyzing historical and real-time healthcare data using AI and machine learning techniques.
- **Healthcare Access Mapping:** Identifying underserved regions and healthcare accessibility gaps through GIS-based mapping and accessibility analysis.
- **Healthcare Resource Management:** Monitoring hospital resources such as bed availability, ICU beds, ambulances, and healthcare facility status.
- **Hospital Data Management:** Allowing hospitals to register, update disease reports, and manage healthcare resources through a centralized platform.
- **Vaccination Campaign Management:** Supporting field workers in conducting vaccination drives, updating vaccination records, and managing healthcare surveys.

- **Health Alerts and Notifications:** Generating early warning alerts and notifications for disease outbreaks and high-risk regions.
- **Mobile Healthcare Unit Management:** Assisting administrators in planning and deploying mobile healthcare units to underserved and high-risk areas.
- **Centralized Data Management:** Maintaining healthcare records, disease reports, vaccination data, healthcare facilities, and regional information in a centralized database.
- **User-Friendly Web Interface:** Providing an easy-to-use web application for Administrators, Hospitals, Field Workers, and Users with secure role-based access.
- **Administrative Management:** Enabling administrators to manage hospitals, diseases, healthcare regions, vaccination campaigns, health alerts, mobile healthcare units, users, and analytical reports efficiently.

1.6 Summary

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system improves public healthcare planning, disease surveillance, and resource management by providing a centralized platform for healthcare monitoring and decision support. By integrating real-time disease reports, healthcare accessibility mapping, AI-based disease trend forecasting, vaccination management, healthcare resource monitoring, and administrative management within a single web application, the proposed solution enables healthcare authorities to detect disease outbreaks early, identify underserved regions, optimize healthcare resource allocation, and respond effectively to public health emergencies. The system promotes data-driven healthcare management and provides a strong foundation for future enhancements in intelligent healthcare systems, predictive analytics, and smart public health infrastructure.

System Analysis

2.1 Identification of Need

2.1.1 System Requirement Analysis

System analysis is a crucial phase in the Software Development Life Cycle (SDLC). It involves studying the existing healthcare management process, identifying the challenges faced by healthcare authorities, hospitals, and field workers, and gathering the necessary requirements to develop an efficient and intelligent healthcare management system. Proper system analysis ensures that the developed system is reliable, secure, scalable, user-friendly, and capable of addressing real-world public healthcare challenges.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system is developed to support healthcare administrators, hospitals, and field workers by providing real-time disease monitoring, healthcare accessibility analysis, AI-based disease forecasting, and healthcare resource management. The system integrates healthcare data, disease reports, vaccination records, regional surveys, and healthcare facility information into a centralized platform to improve public health planning and emergency response. The functional and non-functional requirements of the system have been identified through a detailed analysis of modern healthcare challenges and the need for intelligent decision-support systems.

2.1.2 Study of the Existing System

In the existing healthcare environment, hospitals, healthcare departments, and field workers primarily rely on manual reporting systems and independent healthcare databases to manage disease information and healthcare services. Disease reports, vaccination records, healthcare facility details, and regional health surveys are often maintained separately, making it difficult to obtain a complete picture of public health conditions.

Although government health portals and hospital management systems provide healthcare information, they generally lack AI-based disease forecasting, real-time

healthcare access mapping, and centralized coordination among hospitals, field workers, and healthcare authorities. Healthcare data is distributed across multiple systems, making disease monitoring, resource allocation, and emergency response slow and inefficient.

As a result, healthcare authorities face challenges such as delayed disease outbreak detection, inefficient deployment of healthcare resources, poor identification of underserved regions, and difficulty in planning vaccination campaigns. These limitations highlight the need for an intelligent and integrated healthcare management platform capable of providing predictive analytics and real-time healthcare monitoring.

2.1.3 Existing System

The current healthcare management system mainly depends on conventional reporting methods and fragmented healthcare information systems. Disease surveillance is largely reactive, where healthcare authorities respond only after a significant increase in reported cases. Healthcare facilities independently manage patient records, disease reports, and healthcare resources without effective integration into a centralized platform.

Existing healthcare applications mainly focus on maintaining hospital records or displaying statistical reports. They do not provide intelligent disease prediction, GIS-based healthcare accessibility analysis, mobile healthcare unit planning, or AI-driven decision support. Information collected from hospitals, field workers, vaccination campaigns, and regional surveys is often stored in separate systems, resulting in delayed communication and inefficient healthcare planning.

Overall, the existing system lacks centralized healthcare data integration, predictive intelligence, and real-time monitoring capabilities. These limitations affect healthcare accessibility, disease surveillance, resource utilization, and emergency response, creating the need for an integrated AI-based healthcare management system.

2.1.4 Limitations of the Existing System

The existing healthcare management system suffers from several limitations that reduce its overall efficiency and effectiveness.

- Manual maintenance of disease reports and healthcare records.
- Lack of a centralized healthcare database for integrating hospitals, field workers, and healthcare authorities.
- Delayed disease reporting and outbreak detection.
- Absence of AI-based disease trend forecasting and predictive analytics.
- Limited healthcare access mapping and identification of underserved regions.
- Inefficient allocation and monitoring of healthcare resources such as hospital beds, ICU facilities, and mobile healthcare units.
- Poor coordination and communication among hospitals, field workers, and health administrators.
- Difficulty in managing vaccination campaigns and regional health surveys.
- Lack of real-time monitoring, health alerts, and decision-support mechanisms.
- Security and privacy concerns due to fragmented and disconnected healthcare information systems.

2.1.5 Proposed System

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system introduces an intelligent, centralized, and AI-driven healthcare management platform designed to improve disease surveillance, healthcare accessibility, and healthcare resource management. The system integrates disease reports, healthcare facility information, vaccination records, regional survey data, and AI-based predictive analytics into a single web application.

The platform enables hospitals to report disease cases and update healthcare resources, field workers to conduct regional surveys and vaccination campaigns, and administrators to monitor disease trends, manage healthcare facilities, generate health alerts, and optimize healthcare resource allocation. Developed using Python and the Django Framework, the system ensures reliability, security, scalability, and efficient processing of healthcare data.

Key Features of the Proposed System

- **User Registration and Authentication** – Secure login for Administrators, Hospitals, Field Workers, and Users.
- **Hospital Management** – Registration, approval, disease reporting, and healthcare resource management.
- **Disease Trend Forecasting** – AI-based prediction of disease outbreaks using healthcare data.
- **Healthcare Access Mapping** – GIS-based identification of healthcare accessibility gaps.
- **Vaccination Campaign Management** – Planning and monitoring vaccination activities.
- **Mobile Healthcare Unit Management** – Efficient deployment of mobile healthcare services.
- **Health Alerts and Notifications** – Early warning alerts for disease outbreaks and high-risk regions.
- **Administrative Dashboard** – Management of hospitals, healthcare regions, diseases, healthcare resources, vaccination campaigns, users, and reports.

Overall Benefits of the Proposed System:

The proposed system significantly improves healthcare planning by providing real-time disease monitoring, AI-based forecasting, and healthcare accessibility analysis. It enables healthcare authorities to make faster and more informed decisions, optimize healthcare resource allocation, strengthen disease surveillance, and improve emergency response.

The system promotes data-driven healthcare management by integrating hospitals, healthcare facilities, field workers, and administrators into a centralized platform. It enhances public healthcare delivery while improving operational efficiency, healthcare accessibility, and disease prevention.

Advantages of the Proposed System

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting

system offers several advantages over traditional healthcare management systems:

- Provides AI-based disease prediction and forecasting.
- Enables real-time healthcare monitoring and disease surveillance.
- Improves healthcare accessibility through GIS-based mapping.
- Integrates hospitals, field workers, and healthcare authorities into a centralized platform.
- Supports efficient healthcare resource allocation.
- Enhances vaccination campaign planning and monitoring.
- Generates early health alerts for disease outbreaks.
- Improves decision-making through analytical dashboards and reports.
- Ensures secure, role-based access to healthcare information.
- Supports proactive public health planning and emergency response.

2.1.6 Technology Stack

The proposed system is developed using modern web technologies that ensure reliability, scalability, maintainability, and security.

Frontend	HTML, CSS, JavaScript
Backend	Python Django
Database	SQLite3
IDE	Visual Studio Code
Operating System	Windows 10/11
Browser	Google Chrome

Table 2.1: Technology Stack Used

2.2 Project Planning and Scheduling

2.2.1 Introduction

Project planning and scheduling are essential activities in the Software Development Life Cycle (SDLC). They help in organizing project activities, allocating resources, estimating development time, and ensuring the successful implementation of the system. Proper planning ensures that project objectives are achieved within the specified time while maintaining quality, security, reliability, and efficiency.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system was developed through a systematic process that included requirement gathering, system analysis, system design, database design, implementation, AI model integration, testing, and documentation. Each phase was carefully planned and monitored to ensure the successful development of the system. Proper scheduling helped coordinate the development of healthcare management modules such as disease trend forecasting, healthcare access mapping, hospital management, vaccination campaign management, mobile healthcare unit management, health alert generation, and administrative dashboards. This structured approach minimized development risks, optimized resource utilization, and ensured the timely delivery of a reliable, scalable, and intelligent healthcare management platform.

2.2.2 Project Planning

Project planning involves defining project objectives, identifying system requirements, selecting appropriate technologies, and establishing a structured development schedule. For the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system, planning began with studying the existing healthcare management process and analyzing the challenges faced by healthcare authorities, hospitals, and field workers in disease surveillance, healthcare accessibility, resource management, and emergency response.

Based on this analysis, the project scope and objectives were clearly defined. Python and the Django Framework were selected as the primary development technologies due

to their reliability, scalability, security, and ease of implementation. SQLite was selected as the database management system for storing healthcare records, disease reports, hospital information, vaccination data, healthcare resources, and user information. Proper planning ensured that all project activities remained aligned with the identified requirements and objectives while supporting the successful development of an intelligent, AI-powered healthcare management platform.

2.2.3 Project Objectives

The primary objective of the project was to develop an intelligent healthcare management platform capable of monitoring disease trends, mapping healthcare accessibility, and providing AI-based disease forecasting. The system was designed to collect healthcare data from hospitals, field workers, and healthcare authorities, analyze disease patterns, and support informed decision-making for public health management.

Another objective was to improve healthcare accessibility and optimize healthcare resource allocation by identifying underserved regions, monitoring hospital resources, managing vaccination campaigns, and generating early health alerts. The project also aimed to provide healthcare administrators with analytical dashboards and reports for effective planning, monitoring, and emergency response.

2.2.4 Project Scheduling

Project scheduling involves determining the sequence of development activities and estimating the time required to complete each task. The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system was divided into several phases, including requirement analysis, system design, database design, AI model development, application implementation, testing, deployment, and documentation.

Each phase was assigned a specific duration and milestone to ensure smooth project execution. The scheduling process helped manage dependencies between activities and ensured the timely completion of project deliverables.

2.2.5 Resource Planning

Resource planning involves identifying the hardware, software, and human resources required for project development and deployment.

The hardware requirements included computers with sufficient processing power, memory, storage capacity, and internet connectivity. The software resources included Python, Django, SQLite, HTML, CSS, JavaScript, Visual Studio Code, and AI/Machine Learning libraries for disease prediction and healthcare analytics.

Human resources involved in the project included developers, project guides, healthcare domain experts, AI/ML specialists, testers, hospital representatives, field workers, and end users who contributed during various stages of analysis, development, testing, and evaluation.

2.2.6 Risk Analysis

Risk analysis helps identify potential issues that may affect the successful completion and operation of the project. Possible risks in the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system included inaccurate healthcare data, AI prediction errors, delayed disease reporting, database failures, software bugs, cybersecurity threats, and network connectivity issues.

To minimize these risks, secure authentication mechanisms, proper data validation techniques, regular software testing, AI model evaluation, database backup strategies, and data encryption methods were implemented. Continuous monitoring and validation helped identify and resolve issues at an early stage.

2.2.6 Time Management

Time management plays an important role in ensuring successful project completion. The development timeline was divided into multiple phases, with each phase assigned a specific duration based on project complexity and resource availability.

Adequate time was allocated for system analysis, database design, AI model development, coding, testing, debugging, deployment, and documentation. Project progress was reviewed regularly to ensure that all activities were completed according to the planned schedule.

2.2.7 Project Monitoring and Control

Project monitoring and control involve tracking development activities and comparing actual progress against planned objectives. Regular project reviews, testing reports, AI model evaluation results, system performance analysis, and progress assessments were conducted throughout the development process.

Corrective actions were taken whenever deviations from the project plan were identified. Effective monitoring and control ensured that the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system was completed successfully while maintaining the required standards of accuracy, performance, security, scalability, and reliability.

2.2.9 Gantt Chart

A Gantt chart is a project management technique used for scheduling, planning, and resource allocation. It is represented as a bar chart, where each bar corresponds to a specific project activity. The Gantt chart provides a graphical representation of the project schedule, helping developers plan, coordinate, monitor, and track the progress of different tasks throughout the project lifecycle.

The horizontal axis represents the project timeline, while the vertical axis lists the various development activities such as requirement analysis, system design, database design, AI model development, frontend and backend implementation, healthcare access mapping, disease forecasting, testing, deployment, and documentation. Horizontal bars of varying lengths indicate the start date, duration, and completion time of each activity.

The Gantt chart provides a clear overview of project progress and assists in efficient resource utilization and time management. Although it effectively displays project schedules, it does not represent task dependencies between activities. The Gantt chart for the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting project is shown below.



Figure 2.1: Gantt Chart of AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting

- User Request and approval
- User Requirement Study
- Initial Investigation
- Feasibility Study
- Requirement Specification and Approval
- Detailed Investigation
- Selection of a prototype
- Physical Design
- Logical Design (coding)
- Testing
- Implementation
- User Training

2.2.10 PERT Chart

A Program Evaluation and Review Technique (PERT) Chart is a project management tool used to schedule, organize, and coordinate project activities. It graphically represents the sequence of tasks, showing the dependencies between different phases of the project. Each activity is represented by an event or milestone connected by directional arrows, indicating the order in which tasks must be completed. Unlike the Gantt chart, the PERT chart clearly illustrates task dependencies and helps identify the critical path of the project.

For the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting project, the PERT chart was used to plan and monitor the development process effectively. It helped identify the sequence of major activities such as requirement analysis, system design, database design, AI model development, healthcare access mapping implementation, frontend and backend development, testing, deployment, and documentation. By identifying the critical path, the development team was able to allocate resources efficiently, anticipate potential delays, and implement corrective measures whenever necessary.

The PERT chart also assisted in monitoring project progress throughout the Software Development Life Cycle (SDLC), ensuring that each phase was completed according to the planned schedule while maintaining the required quality, security, and reliability standards. The project commenced in the planned development period and progressed through each milestone in a systematic manner. The PERT chart of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting project is shown below, where the various project activities are represented by their respective activity IDs.

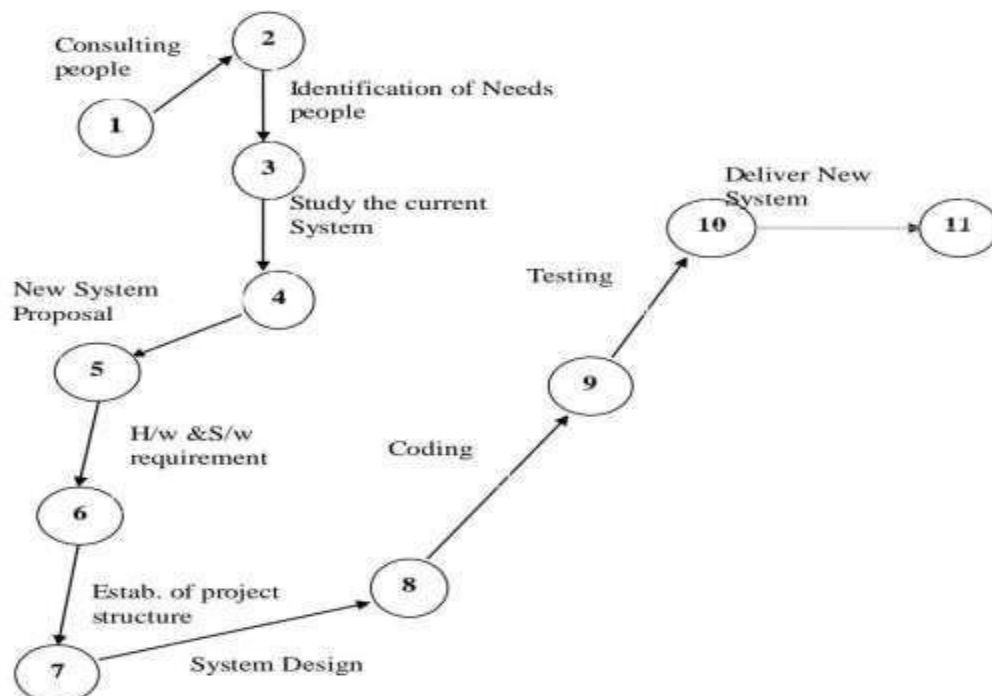


Figure 2.2: PERT Chart of AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting

2.2.11 Summary

Project planning and scheduling provided a structured framework for managing the development of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. Through proper planning, effective resource allocation, risk management, and continuous monitoring, the project was successfully completed within the planned timeframe. Efficient scheduling ensured the timely completion of development activities, while proper utilization of resources and risk mitigation strategies contributed to the system's accuracy, security, reliability, and overall performance.

The planning process established a strong foundation for the subsequent phases of system design, implementation, AI model integration, testing, deployment, and maintenance. By following a systematic Software Development Life Cycle (SDLC) approach, the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system successfully achieved its objectives of providing AI-based disease trend forecasting, healthcare access mapping, healthcare resource management, vaccination campaign monitoring, and real-time decision support for healthcare administrators, hospitals, field workers, and public health authorities.

2.3 Software Requirement Specification

The Software Requirement Specification (SRS) is a formal document that describes the functionalities, requirements, constraints, and expected behavior of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. It serves as a communication bridge between developers, project supervisors, healthcare professionals, government health authorities, hospitals, and other stakeholders.

The SRS defines the system objectives, functional and non-functional requirements, user requirements, database requirements, AI-based disease forecasting requirements, healthcare access mapping requirements, security requirements, and operational constraints. It specifies the features required for disease trend forecasting, healthcare

resource management, hospital management, vaccination campaign management, healthcare access analysis, health alert generation, and administrative monitoring.

A well-defined SRS helps minimize ambiguity, improves development efficiency, and ensures that the final system meets the needs of healthcare administrators, hospitals, field workers, and public health authorities. The document also serves as a reference throughout the Software Development Life Cycle (SDLC), maintaining consistency during system design, implementation, testing, deployment, and future maintenance while ensuring the reliability, scalability, security, and performance of the healthcare management platform.

2.4 Software Engineering Paradigm Applied

The development of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system follows the Agile Software Engineering Paradigm. Agile is an iterative and incremental software development methodology that emphasizes flexibility, collaboration, continuous improvement, and rapid delivery of software components. Unlike traditional development approaches, Agile enables developers to adapt to changing requirements throughout the development lifecycle.

The Agile paradigm was selected because it supports continuous interaction between developers, healthcare professionals, hospitals, field workers, and public health authorities. Features such as user authentication, hospital management, disease trend forecasting, healthcare access mapping, vaccination campaign management, healthcare resource management, health alert generation, and administrative dashboards were developed in multiple iterations. Each module was independently designed, implemented, tested, and refined before being integrated into the complete system.

The Agile development process included requirement gathering, planning, system design, implementation, AI model integration, testing, review, and deployment. Regular feedback was collected during development and incorporated into subsequent iterations. This approach ensured that the final application closely matched user requirements while improving system quality and performance.

The implementation of the Agile methodology provided several benefits, including improved communication among stakeholders, early detection of software defects, reduced development risks, and enhanced software quality. Continuous testing and evaluation ensured reliable system performance, while the flexibility of Agile allowed new features and improvements to be incorporated efficiently. Therefore, the Agile Software Engineering Paradigm served as an effective framework for developing the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system, ensuring reliability, scalability, maintainability, security, and user satisfaction.

2.5 Data models

2.5.1 Data Flow Diagram (DFD)

A Data Flow Diagram (DFD) is a graphical representation used to illustrate the movement of data within a system. It shows how data enters the system, how it is processed, where it is stored, and how it flows between different system components. DFDs are widely used during the system analysis and design phase because they provide a clear understanding of system functionality without focusing on implementation details.

The primary objective of a DFD is to model the flow of information and identify interactions between users, processes, and data stores. It helps developers and stakeholders understand the operational workflow of the system and serves as the foundation for designing efficient software solutions.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system utilizes DFDs to represent the flow of information among Administrators, Hospitals, Field Workers, Users, and the centralized healthcare database. Through DFDs, the movement of data related to user registration, hospital registration, disease reporting, healthcare resource management, healthcare access mapping, AI-based disease forecasting, vaccination campaign management, health alert generation, regional survey management, and administrative monitoring can be clearly visualized. These diagrams help in understanding how information flows throughout the system and ensure that all functional requirements are properly addressed during system development.

The commonly used notations in the DFD are shown below.

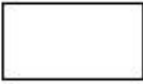
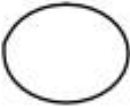
Symbol	Meaning
	An entity. A source of data or a destination for data.
	A process or task that is performed by the system.
	A data store, a place where data is held between processes.
	A data flow.

Figure 2.3: symbols used

DFD Level 0 (Context Diagram)

The Level 0 DFD, also known as the Context Diagram, provides a high-level overview of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. It represents the entire system as a single process and illustrates the interaction between the system and external entities such as Administrators, Hospitals, Field Workers, and Users. This diagram helps stakeholders understand the overall scope, functionality, and boundaries of the system.

Through the Level 0 DFD, the flow of information related to user registration, hospital registration, disease reporting, healthcare resource management, healthcare access mapping, AI-based disease trend forecasting, vaccination campaign management,

health alerts, regional health surveys, and administrative monitoring can be clearly visualized. It shows how hospitals submit disease reports and update healthcare resources, field workers conduct vaccination campaigns and regional surveys, administrators monitor healthcare activities and manage the overall system, and users access healthcare information and services. The Context Diagram illustrates the overall interaction between all stakeholders and the centralized healthcare management system, ensuring efficient communication, real-time monitoring, and effective public health decision-making.

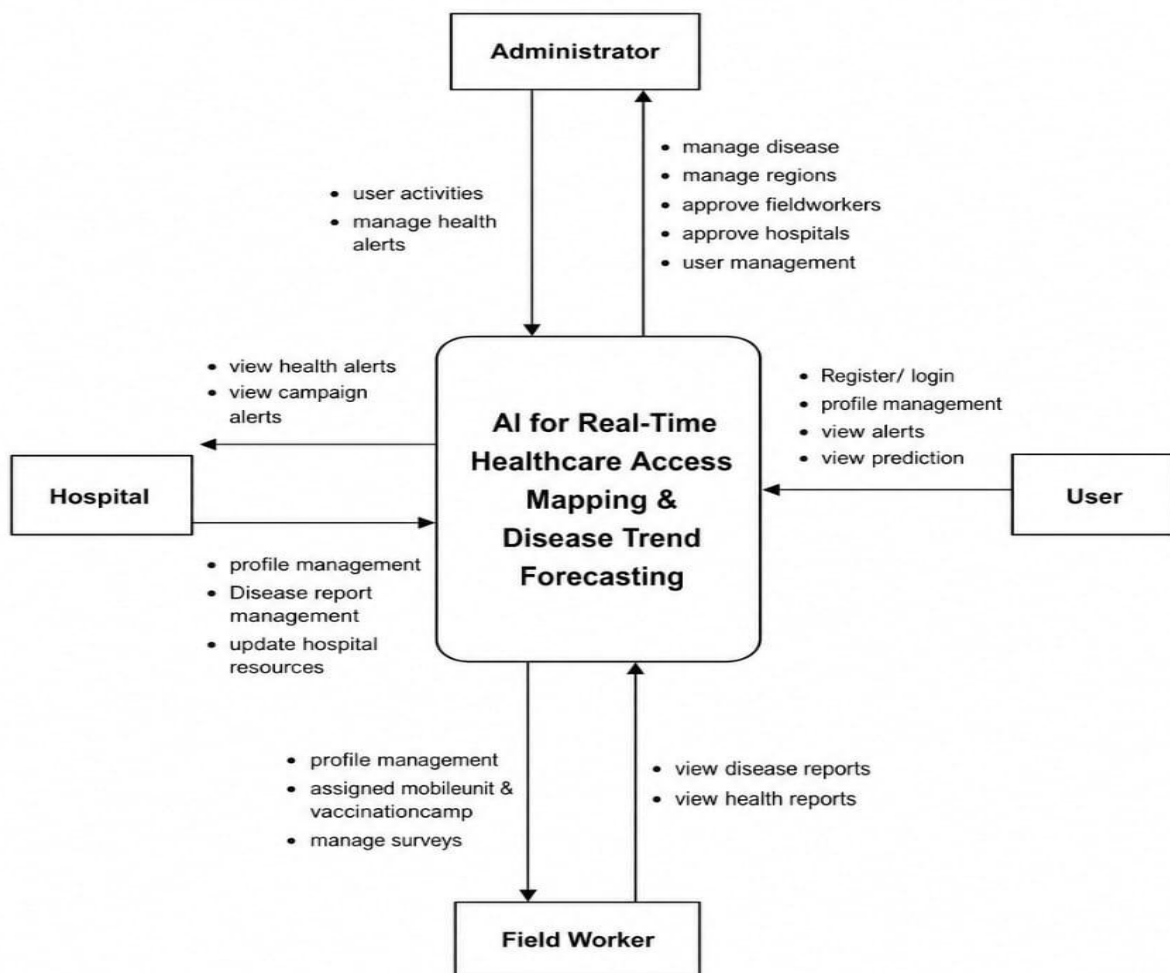


Figure 2.4: DFD Level 0 (Context Diagram)

DFD Level 1

The Level 1 Data Flow Diagram (DFD) illustrates the major functions available to the User Module in the AI for Real-Time Healthcare Access Mapping & Disease Trend Forecasting system. Registered users interact with the application through several services that provide healthcare-related information.

The User Management process allows users to register, log in, and update their profiles, with the information stored in the Registration database. Users can view nearby hospitals through an interactive map, where hospital information is retrieved from the Hospital database. The system also enables users to view health alerts, vaccination campaign notifications, and disease awareness information, which are obtained from the respective Health Alert, Vaccination Campaign, and Disease databases. These services help users stay informed about disease outbreaks, healthcare facilities, and preventive healthcare programs.

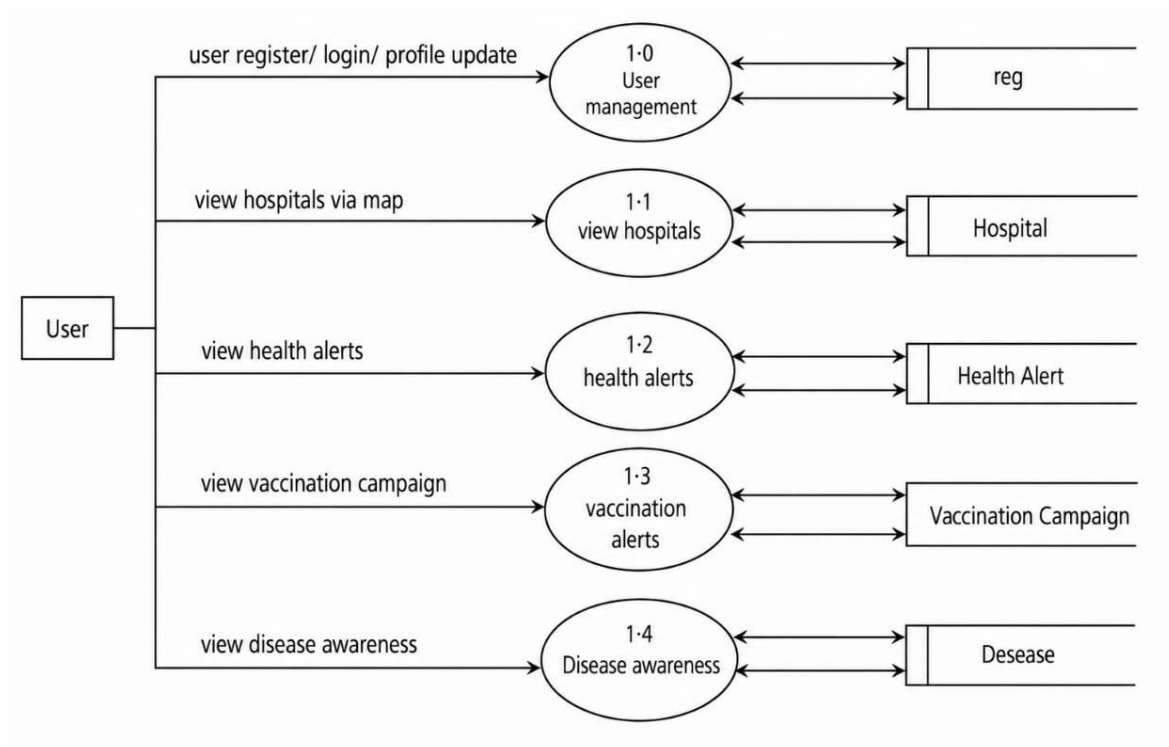


Figure 2.5: DFD Level 1

DFD Level 2

The Level 2 Data Flow Diagram (DFD) represents the functions performed by the Hospital Module. Hospitals are responsible for maintaining healthcare data and updating resource availability within the system.

Hospitals first complete the registration, login, and profile management process. They can add, edit, delete, and maintain disease case reports, which are stored in the Disease database for AI-based disease forecasting and monitoring. Hospitals also update healthcare resources, including ICU bed availability, hospital beds, ambulances, and other medical facilities. Additionally, hospitals can view health alerts and vaccination campaign notifications issued by the administrator. This module ensures that real-time hospital resource information is available to users and healthcare authorities.

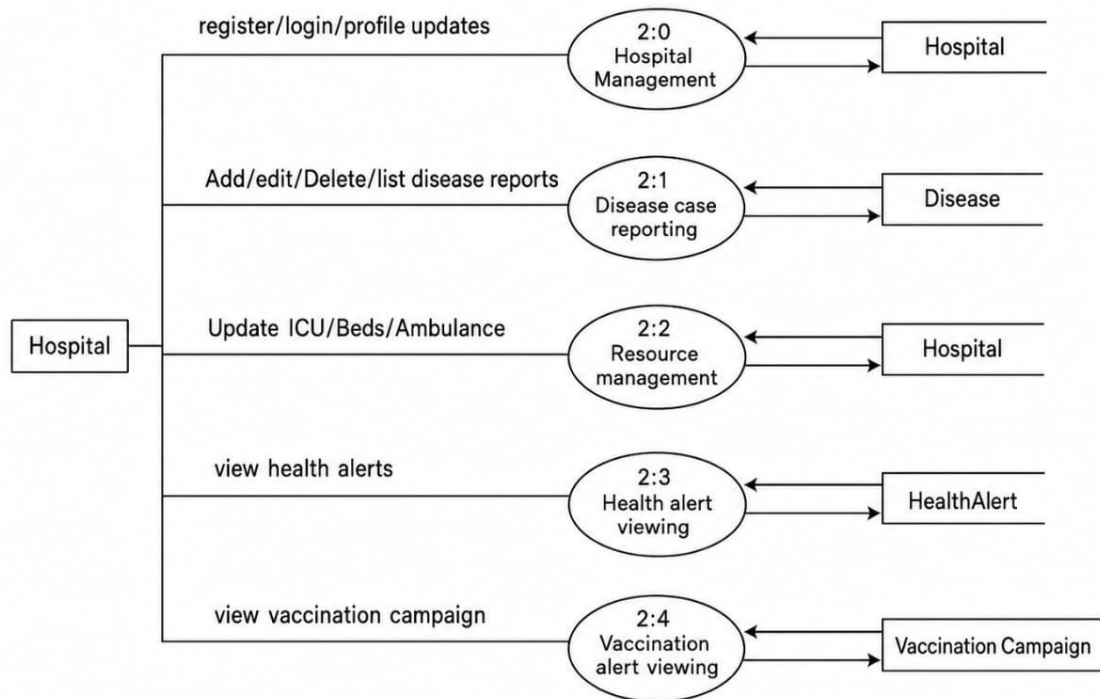


Figure 2.6: DFD Level 2

DFD Level 3

The Level 3 Data Flow Diagram (DFD) illustrates the operations performed by the Field Worker Module. Field workers act as the link between healthcare authorities and the public by collecting field-level healthcare information.

Field workers can register, log in, and manage their profiles after administrator approval. The system assigns mobile healthcare units and vaccination camps to field workers, enabling them to carry out healthcare activities in different regions. They can submit camp completion reports, manage community health surveys, and view disease monitoring reports to identify disease trends. Field workers also receive health alerts from the system to support timely healthcare services. All submitted reports and survey data are stored in their respective databases for AI-based analysis and healthcare planning.

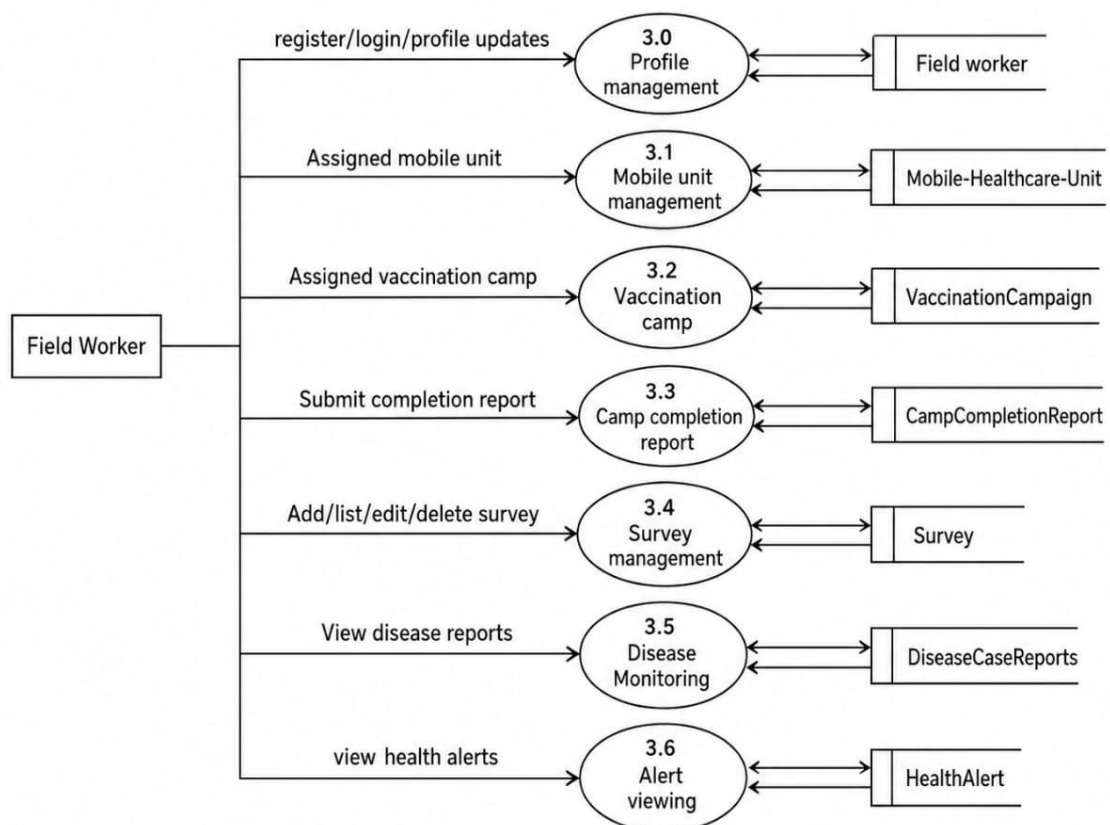


Figure 2.5: DFD Level 3

DFD Level 4

The Level 4 Data Flow Diagram (DFD) describes the administrative functions of the Administrator Module, which has complete control over the AI for Real-Time Healthcare Access Mapping & Disease Trend Forecasting system.

The administrator is responsible for managing users, including viewing and deleting user accounts. The administrator can approve or reject hospital registrations and field worker registrations before granting system access. The system also allows the administrator to manage regions, diseases, mobile healthcare units, vaccination campaigns, and health alerts by performing add, edit, delete, and update operations. Each administrative process interacts with the corresponding database to maintain accurate healthcare information. These functions ensure secure system management, efficient healthcare coordination, and reliable real-time disease monitoring across all regions.

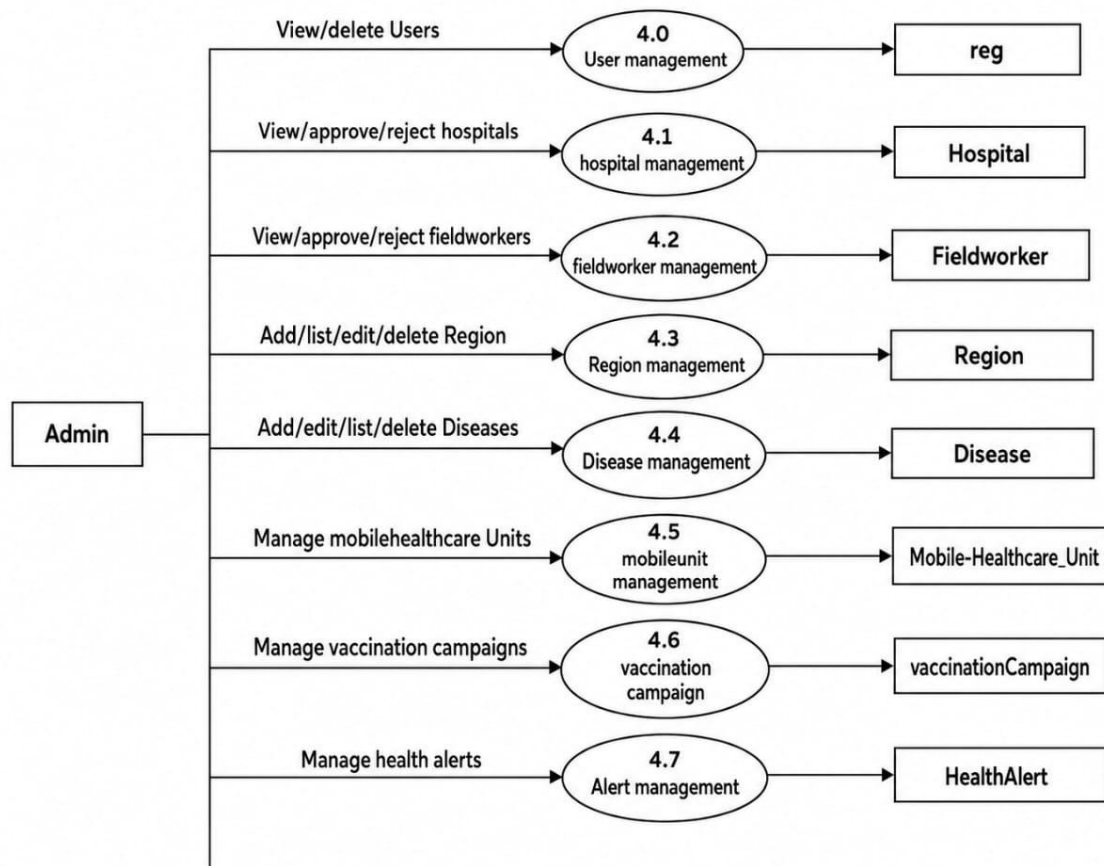


Figure 2.6: DFD Level 4

2.5.2 Entity Relationship Diagram (ER Diagram)

The Entity Relationship (ER) Diagram represents the logical structure of the database used in the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. It illustrates the entities, their attributes, and the relationships among them. The ER diagram helps in understanding how healthcare data such as users, hospitals, diseases, healthcare facilities, vaccination campaigns, health alerts, and disease reports are organized and managed. It serves as the foundation for designing an efficient, secure, and reliable database for the system.flows within the system.

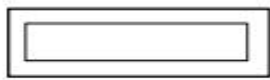
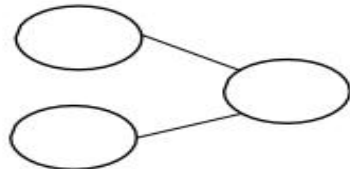
Symbol	Meaning
	Entity
	Relationship
	Attribute
	Weak entity
	Weak entity relationship
	Multivalued Attribute
	Key Attribute
	Composite Attribute

Figure 2.7: Entity Relationship Diagram symbol

The primary entities included in the **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system are **Administrator**, **User**, **Hospital**, **Field Worker**, **Disease**, **Disease Report**, **Healthcare Facility**, **Healthcare Access Score**, **Vaccination Campaign**, **Health Alert**, **Mobile Healthcare Unit**, **Region**, and **Disease Forecast**. These entities are interconnected through one-to-one and one-to-many relationships to ensure efficient data organization and smooth system functionality.

The **User** entity manages user registration, authentication, and profile information. The **Hospital** entity maintains hospital details, healthcare resources, and disease reports submitted by healthcare institutions. The **Field Worker** entity stores information related to field staff, regional surveys, and vaccination activities.

The **Disease** and **Disease Report** entities maintain records of diseases, reported cases, and outbreak details. The **Disease Forecast** entity stores AI-generated predictions and risk levels for future disease outbreaks. The **Healthcare Facility** and **Healthcare Access Score** entities store information about healthcare infrastructure, accessibility, and healthcare coverage across different regions.

The **Vaccination Campaign** entity manages vaccination drives and vaccination records, while the **Health Alert** entity stores alerts generated for high-risk regions. The **Mobile Healthcare Unit** entity manages the deployment of mobile healthcare services to underserved areas. The **Region** entity stores geographical and demographic information used for healthcare access mapping and disease analysis.

The **Administrator** entity is responsible for managing users, hospitals, diseases, healthcare facilities, vaccination campaigns, health alerts, and overall system operations. The ER Diagram provides a clear representation of the relationships among these entities, reduces data redundancy, ensures data integrity through primary and foreign key relationships, and supports efficient data management and retrieval throughout the system.

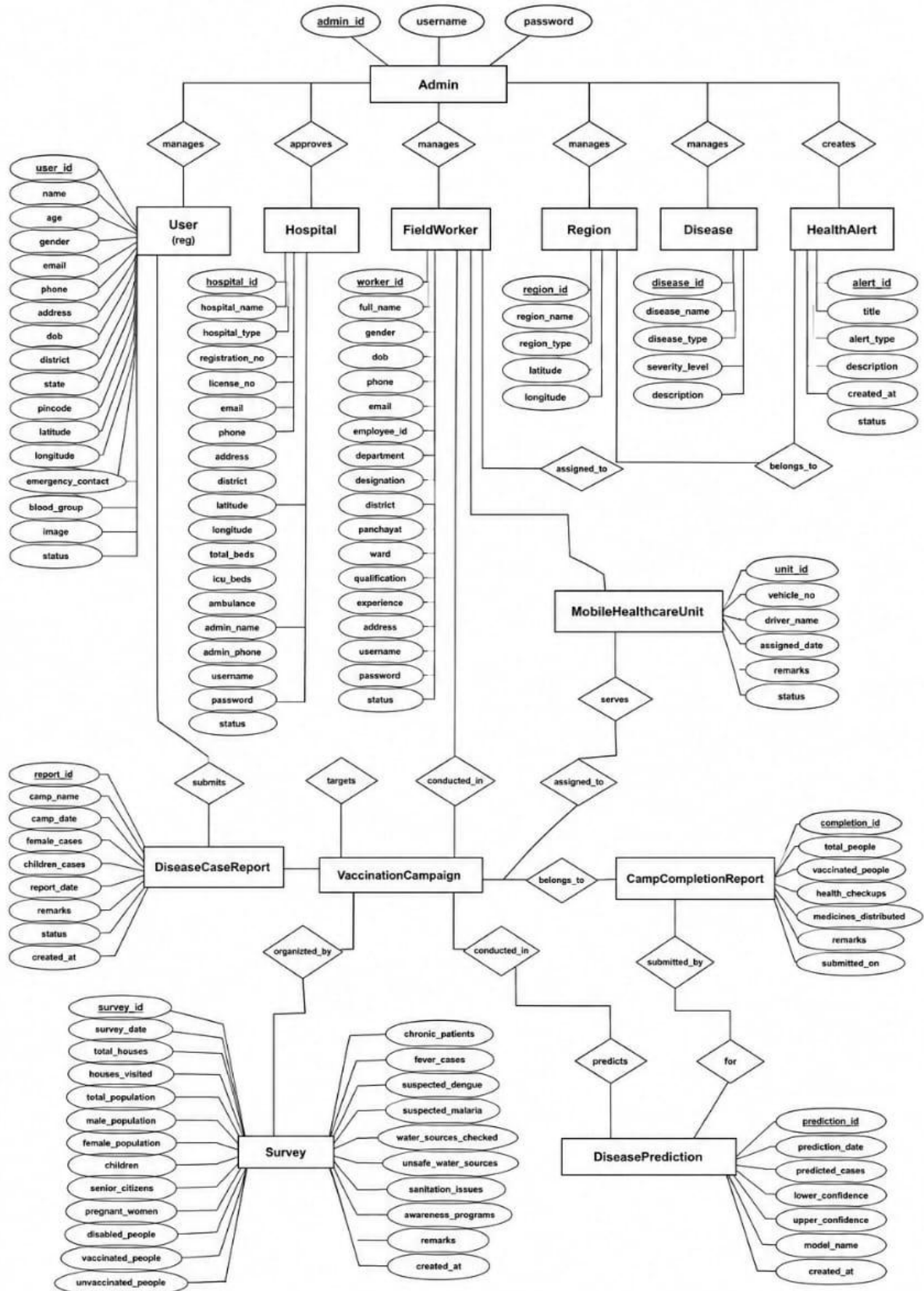


Figure 2.8: Entity Relationship Diagram (ER Diagram)

System Design

3.1 Introduction

System Design is one of the most important phases in the Software Development Life Cycle (SDLC). After gathering and analyzing user requirements, the next step is to transform those requirements into a technical design that can be implemented effectively. System design provides a blueprint for software development by defining the system architecture, modules, database structure, user interfaces, and data flow mechanisms. A well-designed system improves maintainability, scalability, security, and performance while reducing implementation complexity.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system is designed as a centralized web-based platform that supports healthcare authorities in monitoring disease outbreaks, analyzing healthcare accessibility, forecasting disease trends, and managing healthcare resources. The system integrates multiple functionalities including user management, hospital management, disease reporting, healthcare access mapping, AI-based disease forecasting, vaccination campaign management, mobile healthcare unit management, health alert generation, and administrative monitoring into a single platform. This integration improves public healthcare planning, disease surveillance, and emergency response.

The design phase focuses on identifying system components, defining relationships among modules, designing the database structure, and developing user-friendly interfaces that satisfy stakeholder requirements. The design also ensures that the system can support future enhancements without significant architectural modifications.

3.2 Modularized Design

3.2.1 Administrative and Analytics Module

This module is designed for healthcare administrators and public health authorities. It provides administrative controls, analytical dashboards, healthcare reports, disease trend visualization, and system monitoring tools. Administrators can manage users,

hospitals, healthcare regions, diseases, healthcare facilities, vaccination campaigns, mobile healthcare units, and health alerts while analyzing healthcare statistics for better decision-making.

3.2.2 User Management Module

This module serves as the entry point of the system where Users, Hospitals, Field Workers, and Administrators register and access the platform through secure authentication. It manages user profiles, role-based access control, login, password management, and profile updates, ensuring secure access to the healthcare management system.

3.2.3 Healthcare Data Collection and Management Module

This module is responsible for collecting, organizing, and maintaining healthcare-related information from hospitals, field workers, and healthcare authorities. It manages disease reports, healthcare facilities, hospital resources, regional survey data, vaccination records, and demographic information. The module ensures that healthcare data is securely stored and readily available for analysis.

3.2.4 AI-Based Disease Trend Forecasting Module

This is the core module of the system. It utilizes Artificial Intelligence and Machine Learning algorithms to analyze historical and real-time healthcare data for predicting disease outbreaks and identifying high-risk regions. The module generates disease forecasts and risk levels, enabling healthcare authorities to take preventive measures and improve public health planning.

3.2.5 Healthcare Access Mapping Module

This module analyzes the availability and accessibility of healthcare facilities across different regions using GIS technology. It identifies underserved areas, evaluates healthcare accessibility scores, and assists administrators in planning healthcare services and deploying mobile healthcare units to regions requiring immediate attention.

3.2.6 Vaccination and Health Alert Module

This module manages vaccination campaigns, vaccination records, and health alert notifications. Field workers can update vaccination progress, while the system generates health alerts and early warning notifications for disease outbreaks and high-risk areas to support timely public health interventions.

3.2.7 User Interface and Database Management Module

This module provides a user-friendly web interface for Administrators, Hospitals, Field Workers, and Users to interact with the system efficiently. It also manages the centralized database by securely storing healthcare records, disease reports, healthcare resources, vaccination data, healthcare accessibility information, and AI prediction results. The module ensures data security, backup, access control, and efficient retrieval of information for system operations.

3.3 Data Integrity Constraints

Data Integrity Constraints are rules applied to the database to ensure the accuracy, consistency, and reliability of healthcare information stored in the **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system. These constraints prevent invalid data entry and maintain proper relationships among database tables.

3.3.1 Entity Integrity

Entity Integrity ensures that each table contains a unique identifier known as the **Primary Key**. The primary key cannot be **NULL** and uniquely identifies every record.

Examples of primary keys used in the system include:

- User ID in the User table
- Hospital ID in the Hospital table
- Field Worker ID in the Field Worker table
- Disease Report ID in the Disease Report table

- Vaccination Campaign ID in the Vaccination Campaign table
- Health Alert ID in the Health Alert table

Entity integrity prevents duplicate records and ensures each record can be uniquely identified.

3.3.2 Referential Integrity

Referential Integrity maintains consistency between related tables using **Foreign Keys**.

For example:

- A Disease Report must reference an existing Hospital.
- A Vaccination Record must reference an existing Vaccination Campaign.
- A Field Worker must belong to an existing Region.
- A Health Alert must be associated with an existing Disease.

These constraints ensure valid relationships among database tables.

3.3.3 Domain Integrity

Domain Integrity ensures that data entered into a field follows predefined rules such as data type, format, length, and acceptable values.

Examples include:

- Email must contain a valid email address.
- Phone number must contain only numeric values.
- Disease Risk Level must be **Low**, **Medium**, or **High**.
- User Role must be **Administrator**, **Hospital**, **Field Worker**, or **User**.

Domain integrity improves data accuracy and minimizes input errors.

3.3.4 NOT NULL Constraint

The **NOT NULL** constraint ensures that mandatory fields cannot be left empty.

Examples include:

- Full Name
- Username

- Password
- Hospital Name
- Disease Name
- Region Name

This ensures essential information is always available.

3.3.5 UNIQUE Constraint

The **UNIQUE** constraint prevents duplicate values.

Examples include:

- Username
- Email Address
- Mobile Number
- Hospital Registration Number

This ensures every user and hospital is uniquely identified.

3.3.6 CHECK Constraint

The **CHECK** constraint restricts values to predefined conditions.

Examples include:

- Bed Availability must be greater than or equal to zero.
- ICU Beds cannot be negative.
- Vaccination Count cannot be negative.
- Disease Risk Score must be between **0 and 100**.

These constraints improve data validity and consistency.

3.3.7 Default Constraints

Default constraints automatically assign values when users do not provide input.

Examples include:

- Registration Date defaults to the current date.
- Hospital Status defaults to **Pending**.

- User Status defaults to **Active**.
- Health Alert Status defaults to **Active**.

These defaults simplify data entry and maintain consistency.

3.3.8 Data Validation Rules

Additional validation rules are implemented using **Django Models** and **Forms**.

These validations include:

- Email format verification.
- Password strength validation.
- Mobile number length verification.
- Healthcare resource validation.
- Disease report validation.
- Date and time format validation.

Application-level validation works together with database constraints to ensure complete data integrity.

3.3.9 Advantages of Data Integrity Constraints

The implementation of data integrity constraints provides several benefits:

- Prevents duplicate and invalid records.
- Maintains consistency among related tables.
- Improves data accuracy and reliability.
- Reduces database anomalies.
- Enhances system security and performance.
- Supports efficient healthcare data management and decision-making.

Data integrity constraints play a vital role in ensuring that the **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system maintains accurate, consistent, secure, and reliable healthcare information throughout its operation.

3.4 Database Design

3.4.1 Introduction

Database design is one of the most critical phases in software development. It involves the systematic organization of data to ensure efficient storage, retrieval, modification, and management of information. A properly designed database improves system performance, reduces redundancy, maintains data integrity, and enhances overall reliability.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system manages a large amount of healthcare information related to users, hospitals, field workers, diseases, disease reports, healthcare facilities, vaccination campaigns, healthcare resources, health alerts, regions, mobile healthcare units, and AI-based disease forecasts. Therefore, a centralized database is required to store and manage these records efficiently. The database supports various operations such as user registration, hospital management, disease reporting, healthcare resource monitoring, healthcare access mapping, vaccination management, AI-based disease forecasting, health alert generation, and administrative monitoring.

The database for the proposed system is designed using SQLite and integrated with the Django Framework. The database structure is normalized to minimize redundancy and maintain data consistency. Relationships among tables are established using primary keys and foreign keys, enabling efficient data retrieval and management. The database design ensures secure storage of healthcare information and provides a strong foundation for the smooth operation, scalability, and future enhancement of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system.

3.4.2 Table Design

Table 3.1: reg (User)

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key, Auto Increment	User ID
name	CharField	50	NOT NULL	User name
age	PositiveIntegerField	-	NOT NULL	User age
gender	CharField	1	Choices	Gender
email	EmailField	254	UNIQUE	Email address
password	CharField	100	NOT NULL	Password
phone	CharField	10	NOT NULL	Mobile number
image	ImageField	-	NULL	User photo
address	TextField	-	NOT NULL	Address
dob	DateField	-	NOT NULL	Date of birth
district	CharField	30	Choices	District
state	CharField	30	Choices	State
pincode	CharField	6	NOT NULL	PIN Code
latitude	DecimalField	9,6	NULL	Latitude
longitude	DecimalField	9,6	NULL	Longitude
emergency_contact	CharField	10	NOT NULL	Emergency contact
blood_group	CharField	5	Choices	Blood group
status	CharField	10	Default=Active	Account status

Table 3.2: Hospital

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Hospital ID
hospital_name	CharField	255	NOT NULL	Hospital name
hospital_type	CharField	50	Choices	Hospital type
registration_no	CharField	100	UNIQUE	Registration number
license_no	CharField	100	UNIQUE	License number
email	EmailField	254	UNIQUE	Email
phone	CharField	15	NOT NULL	Phone number
address	TextField	-	NOT NULL	Address
district	CharField	100	NOT NULL	District
latitude	DecimalField	9,6	NULL	Latitude
longitude	DecimalField	9,6	NULL	Longitude

Field Name	Data Type	Size	Constraint	Description
total_beds	CharField	10	NULL	Total beds
icu_beds	CharField	10	NULL	ICU beds
ambulance	BooleanField	-	Default=False	Ambulance availability
admin_name	CharField	150	NOT NULL	Admin name
admin_phone	CharField	15	NOT NULL	Admin phone
username	CharField	100	UNIQUE	Login username
password	CharField	128	NOT NULL	Password
status	CharField	10	Default=Pending	Approval status
created_at	DateTimeField	-	Auto	Created date
updated_at	DateTimeField	-	Auto	Updated date

Table 3.3: Region

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Region ID
region_name	CharField	100	NOT NULL	Region name
region_type	CharField	20	Choices	Urban/Rural/Tribal
latitude	DecimalField	9,6	NOT NULL	Latitude
longitude	DecimalField	9,6	NOT NULL	Longitude

Table 3.4: Disease

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Disease ID
disease_name	CharField	100	NOT NULL	Disease name
disease_type	CharField	100	NOT NULL	Disease type
severity_level	CharField	20	Choices	Severity
description	TextField	-	NULL	Disease description

Table 3.5: FieldWorker

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Worker ID
full_name	CharField	100	NOT NULL	Worker name
gender	CharField	20	Choices	Gender

Field Name	Data Type	Size	Constraint	Description
dob	DateField	-	NOT NULL	Date of birth
phone	CharField	10	NOT NULL	Phone
email	EmailField	254	UNIQUE	Email
employee_id	CharField	30	UNIQUE	Employee ID
department	CharField	100	Choices	Department
designation	CharField	100	Choices	Designation
district	CharField	50	Choices	District
region	ForeignKey	-	FK → Region	Assigned region
panchayat	CharField	100	NOT NULL	Panchayat
ward	CharField	30	NOT NULL	Ward
qualification	CharField	100	NOT NULL	Qualification
experience	PositiveIntegerField	-	NOT NULL	Experience
address	TextField	-	NOT NULL	Address
username	CharField	100	UNIQUE	Username
password	CharField	100	NOT NULL	Password
status	CharField	20	Default=Pending	Approval status

Table 3.6: HealthAlert

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Alert ID
title	CharField	200	NOT NULL	Alert title
alert_type	CharField	20	Choices	Alert type
description	TextField	-	NOT NULL	Alert description
district	ForeignKey	-	FK → Region	Region
created_at	DateTimeField	-	Auto	Created date
status	CharField	20	Default=Active	Alert status

Table 3.7: Mobile_Healthcare_Unit

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Unit ID
vehicle_no	CharField	30	NOT NULL	Vehicle number
driver_name	CharField	100	NOT NULL	Driver name
field_worker	ForeignKey	-	FK → FieldWorker	Assigned

Field Name	Data Type	Size	Constraint	Description
				worker
region	ForeignKey	-	FK → Region	Assigned region
assigned_date	DateField	-	NOT NULL	Assignment date
remarks	TextField	-	NOT NULL	Remarks
status	CharField	20	Default=Assigned	Status

Table 3.8: DiseaseCaseReport

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Report ID
hospital	ForeignKey	-	FK → Hospital	Hospital
disease	ForeignKey	-	FK → Disease	Disease
region	ForeignKey	-	FK → Region	Region
case_count	PositiveIntegerField	-	NOT NULL	Total cases
male_cases	PositiveIntegerField	-	Default=0	Male cases
female_cases	PositiveIntegerField	-	Default=0	Female cases
children_cases	PositiveIntegerField	-	Default=0	Children cases
report_date	DateField	-	NOT NULL	Report date
remarks	TextField	-	NULL	Remarks
status	CharField	20	Default=Pending	Status
created_at	DateTimeField	-	Auto	Created date

Table 3.9: VaccinationCampaign

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Campaign ID
camp_name	CharField	150	NOT NULL	Camp name
disease	ForeignKey	-	FK → Disease	Disease
region	ForeignKey	-	FK → Region	Region
hospital	ForeignKey	-	FK → Hospital	Hospital
field_worker	ForeignKey	-	FK → FieldWorker	Worker
camp_date	DateField	-	NOT NULL	Camp date
camp_time	TimeField	-	NOT NULL	Camp time

Field Name	Data Type	Size	Constraint	Description
venue	CharField	250	NOT NULL	Venue
target_population	PositiveIntegerField	-	NOT NULL	Target population
description	TextField	-	NOT NULL	Description
status	CharField	20	Default=Scheduled	Status
created_at	DateTimeField	-	Auto	Created date
updated_at	DateTimeField	-	Auto	Updated date

Table 3.10: CampCompletionReport

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Completion ID
campaign	ForeignKey	-	FK → VaccinationCampaign	Campaign
worker	ForeignKey	-	FK → FieldWorker	Worker
total_people	PositiveIntegerField	-	NOT NULL	Total people
vaccinated_people	PositiveIntegerField	-	NOT NULL	Vaccinated people
health_checkups	PositiveIntegerField	-	NOT NULL	Health checkups
medicines_distributed	PositiveIntegerField	-	NOT NULL	Medicines distributed
remarks	TextField	-	NOT NULL	Remarks
submitted_on	DateTimeField	-	Auto	Submission date

Table 3.11: Survey

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Survey ID
worker	ForeignKey	-	FK → FieldWorker	Worker
region	ForeignKey	-	FK → Region	Region
survey_date	DateField	-	NOT NULL	Survey date
total_houses	PositiveIntegerField	-	NOT NULL	Total houses
houses_visited	PositiveIntegerField	-	NOT NULL	Houses visited
total_population	PositiveIntegerField	-	NOT NULL	Total population

Field Name	Data Type	Size	Constraint	Description
male_population	PositiveIntegerField	-	NOT NULL	Male population
female_population	PositiveIntegerField	-	NOT NULL	Female population
children	PositiveIntegerField	-	NOT NULL	Children
senior_citizens	PositiveIntegerField	-	NOT NULL	Senior citizens
pregnant_women	PositiveIntegerField	-	NOT NULL	Pregnant women
disabled_people	PositiveIntegerField	-	NOT NULL	Disabled people
vaccinated_people	PositiveIntegerField	-	NOT NULL	Vaccinated people
unvaccinated_people	PositiveIntegerField	-	NOT NULL	Unvaccinated people
chronic_patients	PositiveIntegerField	-	NOT NULL	Chronic patients
fever_cases	PositiveIntegerField	-	Default=0	Fever cases
suspected_dengue	PositiveIntegerField	-	Default=0	Suspected dengue
suspected_malaria	PositiveIntegerField	-	Default=0	Suspected malaria
water_sources_checked	PositiveIntegerField	-	Default=0	Water sources checked
unsafe_water_sources	PositiveIntegerField	-	Default=0	Unsafe water sources
sanitation_issues	PositiveIntegerField	-	Default=0	Sanitation issues
awareness_programs	PositiveIntegerField	-	Default=0	Awareness programs
remarks	TextField	-	NULL	Remarks
created_at	DateTimeField	-	Auto	Created date

Table 3.12: DiseasePrediction

Field Name	Data Type	Size	Constraint	Description
id	Integer	-	Primary Key	Prediction ID
disease	ForeignKey	-	FK → Disease	Disease

Field Name	Data Type	Size	Constraint	Description
region	ForeignKey	-	FK → Region	Region
prediction_date	DateField	-	NOT NULL	Prediction date
predicted_cases	FloatField	-	NOT NULL	Predicted cases
lower_confidence	FloatField	-	NOT NULL	Lower confidence interval
upper_confidence	FloatField	-	NOT NULL	Upper confidence interval
model_name	CharField	50	Default=ARIMA	Prediction model
created_at	DateTimeField	-	Auto	Created date

3.4.3 Test Results

The testing activities conducted on the **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system produced satisfactory results. All major functionalities operated according to the specified requirements, and no critical defects were identified during the final testing phase.

Minor issues detected during earlier testing stages were resolved through debugging, validation, and retesting procedures. The final version of the application demonstrated reliable performance, accurate AI-based disease trend forecasting, efficient healthcare data management, secure user authentication, and effective interaction among administrators, hospitals, field workers, and users.

The system successfully managed disease reporting, healthcare resource monitoring, healthcare access mapping, vaccination campaign management, health alert generation, and administrative operations. The AI-based forecasting module accurately analyzed healthcare data to predict disease trends and identify high-risk regions, supporting timely public health decision-making.

The test results confirmed that the **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system is stable, reliable, secure, and ready for deployment. The successful completion of testing indicates that the system can effectively support healthcare authorities in improving disease surveillance, optimizing healthcare resource allocation, enhancing healthcare accessibility, and enabling proactive public health management.

Coding

Source code

views.py

```
from decimal import Decimal

from django.shortcuts import render, redirect
from django.http import HttpResponse
from django.contrib import messages

from . import models

# =====
# Registration
# =====

def registration(request):

    if request.method == "POST":

        name = request.POST.get("name")
        age = request.POST.get("age")
        gender = request.POST.get("gender")
        email = request.POST.get("email")
        phone = request.POST.get("phone")
        password = request.POST.get("password")
        image = request.FILES.get("image")
        address = request.POST.get("address")
        dob = request.POST.get("dob")
        district = request.POST.get("district")
        state = request.POST.get("state")
        pincode = request.POST.get("pincode")
```

```
blood_group = request.POST.get("blood_group")
emergency_contact = request.POST.get("emergency_contact")

latitude = request.POST.get("latitude")
longitude = request.POST.get("longitude")

if models.reg.objects.filter(email=email).exists():
    messages.error(request, "Email already exists.")
    return redirect("registration")

try:
    latitude = Decimal(latitude)
    longitude = Decimal(longitude)
except:
    latitude = None
    longitude = None

models.reg.objects.create(
    name=name,
    age=age,
    gender=gender,
    email=email,
    password=password,
    phone=phone,
    image=image,
    address=address,
    dob=dob,
    district=district,
    state=state,
    pincode=pincode,
    latitude=latitude,
    longitude=longitude,
    emergency_contact=emergency_contact,
```

```

        blood_group=blood_group,
    )

    messages.success(request, "Registration Successful")
    return redirect("login")

return render(request, "registration.html")

# =====
# Login
# =====
def login(request):

    if request.method == "POST":

        email = request.POST.get("email")
        password = request.POST.get("password")

        try:
            user = models.reg.objects.get(email=email)

            if user.password == password:

                if user.status == "Active":

                    request.session["user_id"] = user.id
                    request.session["email"] = user.email

                    return redirect("home")

                else:
                    return HttpResponseRedirect(

```

```

        """
        <h3 style='color:red;text-align:center;'>
        Your account has been deleted or deactivated by the
administrator.

        <br><br>
        Please contact the administrator.
        </h3>
        """
    )

    else:
        return HttpResponseRedirect("Invalid Credentials")

    except models.reg.DoesNotExist:
        return HttpResponseRedirect("User not found")

    return render(request, "login.html")

# =====
# Home
# =====
def home(request):
    return render(request, "home.html")

# =====
# Profile
# =====
def profile(request):

    if "email" in request.session:

```

```

        email = request.session["email"]

    try:
        client = models.reg.objects.get(email=email)
        return render(request, "profile.html", {"client": client})

    except models.reg.DoesNotExist:
        return HttpResponse("User not found")

    return HttpResponse("Not found")

# =====
# Edit Profile
# =====
def editprofile(request):

    if "email" in request.session:

        email = request.session["email"]

    try:

        client = models.reg.objects.get(email=email)

        if request.method == "POST":

            client.name = request.POST.get("name")
            client.age = request.POST.get("age")
            client.gender = request.POST.get("gender")
            client.phone = request.POST.get("phone")
            client.address = request.POST.get("address")
            client.password = request.POST.get("password")
            client.state = request.POST.get("state")

```

```

        client.district = request.POST.get("district")
        client.pincode = request.POST.get("pincode")
        client.dob = request.POST.get("dob")
        client.blood_group = request.POST.get("blood_group")
        client.emergency_contact =
request.POST.get("emergency_contact")

        latitude = request.POST.get("latitude")
        longitude = request.POST.get("longitude")

        try:
            client.latitude = Decimal(latitude)
            client.longitude = Decimal(longitude)
        except:
            pass

        image = request.FILES.get("image")
        if image:
            client.image = image

        client.save()

        messages.success(request, "Profile Updated Successfully.")
        return redirect("profile")

    return render(request, "editprofile.html", {"client": client})

except models.reg.DoesNotExist:
    return HttpResponse("User not found")

return HttpResponse("Not found")

```

```

# =====
# Logout
# =====

def logout(request):

    if "email" in request.session:
        request.session.flush()

    return redirect("index")


from django.shortcuts import render, redirect, get_object_or_404
from django.http import HttpResponseRedirect
from django.db.models import Count
from django.contrib import messages
import json


from . import models
from .models import (
    reg,
    Hospital,
    FieldWorker,
    Region,
    Disease,
    DiseaseCaseReport,
    HealthAlert,
)

# =====
# Admin Login
# =====

def adminlogin(request):

    if request.method == "POST":

```

```

        username = request.POST.get("username")
        password = request.POST.get("password")

        if username == "admin" and password == "admin":

            request.session["admin_logged_in"] = True
            request.session["admin_email"] = username

            return redirect("adminhome")

        messages.error(request, "Invalid Credentials")

    return render(request, "admin_login.html")

# =====
# Admin Home Dashboard
# =====

def adminhome(request):

    if not request.session.get("admin_logged_in"):
        return redirect("adminlogin")

    # -----
    # Dashboard Cards
    # -----

    total_users = reg.objects.count()

    approved_hospitals = Hospital.objects.filter(
        status="Approved"
    ).count()

```

```

pending_hospitals = Hospital.objects.filter(
    status="Pending"
).count()

total_fieldworkers = FieldWorker.objects.count()

total_regions = Region.objects.count()

total_diseases = Disease.objects.count()

total_reports = DiseaseCaseReport.objects.count()

total_alerts = HealthAlert.objects.count()

# -----
# Hospital Beds & ICU Graph
# -----

hospitals = Hospital.objects.filter(status="Approved")

hospital_names = []

bed_counts = []

icu_counts = []

for hospital in hospitals:

    hospital_names.append(hospital.hospital_name)

    if hospital.total_beds and str(hospital.total_beds).isdigit():
        bed_counts.append(int(hospital.total_beds))

```

```

else:
    bed_counts.append(0)

if hospital.icu_beds and str(hospital.icu_beds).isdigit():
    icu_counts.append(int(hospital.icu_beds))
else:
    icu_counts.append(0)

# -----
# Hospital Type Graph
# -----

government = Hospital.objects.filter(
    hospital_type="Government"
).count()

private = Hospital.objects.filter(
    hospital_type="Private"
).count()

clinic = Hospital.objects.filter(
    hospital_type="Clinic"
).count()

medical = Hospital.objects.filter(
    hospital_type="Medical College"
).count()

other = Hospital.objects.filter(
    hospital_type="Other"
).count()

# -----

```

```

# Ambulance Graph
# -----

ambulance_yes = Hospital.objects.filter(
    ambulance=True
).count()

ambulance_no = Hospital.objects.filter(
    ambulance=False
).count()

# -----
# District Graph
# -----

district_data = (
    Hospital.objects.values("district")
    .annotate(total=Count("id"))
)

district_names = []

district_count = []

for item in district_data:
    district_names.append(item["district"])
    district_count.append(item["total"])

# -----
# Context
# -----

context = {

```

```
# Dashboard Cards
"total_users": total_users,
"approved_hospitals": approved_hospitals,
"pending_hospitals": pending_hospitals,
"total_fieldworkers": total_fieldworkers,
"total_regions": total_regions,
"total_diseases": total_diseases,
"total_reports": total_reports,
"total_alerts": total_alerts,
```

```
# Graph 1
"hospital_graph": json.dumps([
    approved_hospitals,
    pending_hospitals
]),
```

```
# Graph 2
"dashboard_graph": json.dumps([
    total_users,
    approved_hospitals,
    total_fieldworkers
]),
```

```
# Graph 3
"resource_graph": json.dumps([
    total_regions,
    total_diseases,
    total_reports,
    total_alerts
]),
```

```
# Graph 4
```

```

        "hospital_names": json.dumps(hospital_names),
        "bed_counts": json.dumps(bed_counts),

        # Graph 5
        "icu_counts": json.dumps(icu_counts),

        # Graph 6
        "hospital_type": json.dumps([
            government,
            private,
            clinic,
            medical,
            other
        ]),

        # Graph 7
        "ambulance_data": json.dumps([
            ambulance_yes,
            ambulance_no
        ]),

        # Graph 8
        "district_names": json.dumps(district_names),
        "district_count": json.dumps(district_count),
    }

    return render(request, "adminhome.html", context)

# =====
# User List
# =====

def userlist(request):

```

```

        users = models.reg.objects.all()

    return render(
        request,
        "userlist.html",
        {
            "i": users
        }
    )

# =====
# Delete User
# =====
def delete_user(request, id):

    user = get_object_or_404(models.reg, id=id)

    user.status = "Deleted"

    user.save()

    return redirect("userlist")

# =====
# Activate User
# =====
def activate_user(request, id):

    user = get_object_or_404(reg, id=id)

```

```

        user.status = "Active"

        user.save()

        return redirect("userlist")

# =====
# Hospital List
# =====

def doctor_list(request):

    doctors = Hospital.objects.all()

    return render(
        request,
        "doctor_list.html",
        {
            "doctors": doctors
        }
    )

# =====
# Approve Hospital
# =====

def approve_doctor(request, id):

    doctor = get_object_or_404(Hospital, id=id)

    doctor.status = "Approved"

    doctor.save()

```

```

        return redirect("doctor_list")

# =====
# Reject Hospital
# =====
def reject_doctor(request, id):

    doctor = get_object_or_404(Hospital, id=id)

    doctor.status = "Rejected"

    doctor.save()

    return redirect("doctor_list")

# =====
# Admin Logout
# =====
def adminlogout(request):

    request.session.flush()

    return redirect("adminlogin")

from .arima import predict_cases

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import os

```

```

def disease_prediction(request):

    diseases = Disease.objects.all()
    regions = Region.objects.all()

    if request.method == "POST":

        disease_id = request.POST.get("disease")
        region_id = request.POST.get("region")

        disease = Disease.objects.get(id=disease_id)
        region = Region.objects.get(id=region_id)

        reports = DiseaseCaseReport.objects.filter(
            disease=disease,
            region=region
        ).order_by("report_date")

        if reports.count() < 10:

            messages.error(
                request,
                "Minimum 10 disease reports required for ARIMA prediction."
            )

            return redirect("disease_prediction")

    # Convert queryset to dataframe
    df = pd.DataFrame(
        list(
            reports.values(

```

```

        "report_date",
        "case_count"
    )
)
)

# ARIMA Prediction
result = predict_cases(df)

actual = result["actual"]
predicted = result["predicted"]
confidence = result["confidence"]

# Delete previous prediction
DiseasePrediction.objects.filter(
    disease=disease,
    region=region
).delete()

prediction_data = []

next_date = actual.index.max()
    # Save Prediction

for i in range(len(predicted)):

    next_date = next_date + pd.Timedelta(days=1)

    predicted_case = float(predicted.iloc[i])

    lower = float(confidence.iloc[i, 0])

    upper = float(confidence.iloc[i, 1])

```

```

# =====
# Prediction Level
# =====

if predicted_case < 20:
    level = "Low"

elif predicted_case < 50:
    level = "Medium"

else:
    level = "High"

# Save to Database

DiseasePrediction.objects.create(

    disease=disease,

    region=region,

    prediction_date=next_date,

    predicted_cases=predicted_case,

    prediction_level=level,

    lower_confidence=lower,

    upper_confidence=upper,

    model_name="ARIMA"

```

```

    )

    # Send to Template

    prediction_data.append({

        "date": next_date,

        "cases": round(predicted_case, 2),

        "level": level,

        "lower": round(lower, 2),

        "upper": round(upper, 2)

    })

    # =====

# CREATE GRAPH
# =====

plt.figure(figsize=(12, 5))

# Actual Cases
plt.plot(
    actual.index,
    actual.values,
    marker="o",
    linewidth=2,
    label="Actual Cases"
)

```

```

# Future Dates
future_dates = pd.date_range(
    start=actual.index.max() + pd.Timedelta(days=1),
    periods=len(predicted)
)

# Predicted Cases
plt.plot(
    future_dates,
    predicted.values,
    marker="o",
    linestyle="--",
    linewidth=2,
    label="Predicted Cases"
)

# Confidence Interval
plt.fill_between(
    future_dates,
    confidence.iloc[:, 0],
    confidence.iloc[:, 1],
    alpha=0.2,
    label="Confidence Interval"
)

# Format Date
plt.gca().xaxis.set_major_formatter(
    mdates.DateFormatter("%d-%b")
)

plt.gca().xaxis.set_major_locator(
    mdates.DayLocator(interval=3)
)

```

```

plt.xticks(rotation=45)

plt.xlabel("Date")
plt.ylabel("Predicted Cases")

plt.title(
    f"ARIMA Disease Prediction\n"
    f"{disease.disease_name} - {region.region_name}"
)

plt.grid(True)

plt.legend()

plt.tight_layout()

# =====
# SAVE GRAPH
# =====

graph_folder = os.path.join(
    settings.BASE_DIR,
    "static",
    "graphs"
)

os.makedirs(
    graph_folder,
    exist_ok=True
)

graph_path = os.path.join(

```

```

        graph_folder,
        "prediction.png"
    )

    plt.savefig(
        graph_path,
        dpi=300,
        bbox_inches="tight"
    )

    plt.close()

# =====
# RETURN RESULT
# =====

    return render(
        request,
        "prediction_result.html",
        {
            "prediction": prediction_data,
            "disease": disease,
            "region": region,
            "graph": "graphs/prediction.png"
        }
    )

    return render(
        request,
        "prediction_form.html",
        {
            "diseases": diseases,
            "regions": regions

```

```
    }  
)
```

```
from .models import DiseasePrediction, Disease, Region
```

```
from django.db.models import Q
```

```
from django.shortcuts import render, redirect
```

```
from .models import DiseasePrediction, Disease, Region
```

```
def prediction_history(request):
```

```
    predictions = DiseasePrediction.objects.select_related(  
        "disease",  
        "region"  
    ).order_by("-prediction_date")
```

```
    diseases = Disease.objects.all()
```

```
    regions = Region.objects.all()
```

```
    disease = request.GET.get("disease")
```

```
    region = request.GET.get("region")
```

```
    level = request.GET.get("level")
```

```
    if disease:
```

```
        predictions = predictions.filter(
            disease_id=disease
        )

    if region:

        predictions = predictions.filter(
            region_id=region
        )

    if level:

        predictions = predictions.filter(
            prediction_level=level
        )

    context = {

        "predictions": predictions,

        "diseases": diseases,

        "regions": regions,

        "selected_disease": disease,

        "selected_region": region,

        "selected_level": level,

    }

    return render(
```

```

        request,
        "prediction_history.html",
        context
    )

def delete_prediction(request, id):

    DiseasePrediction.objects.get(
        id=id
    ).delete()

    return redirect(
        "prediction_history"
    )


from django.db.models import Count, Avg, Max
from .models import DiseasePrediction
import matplotlib.pyplot as plt
import os
from django.conf import settings
import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt

def prediction_dashboard(request):

    predictions = DiseasePrediction.objects.all()

    total_predictions = predictions.count()

```

```
high_count = predictions.filter(
    prediction_level="High"
).count()
```

```
medium_count = predictions.filter(
    prediction_level="Medium"
).count()
```

```
low_count = predictions.filter(
    prediction_level="Low"
).count()
```

```
average_cases = predictions.aggregate(
    Avg("predicted_cases")
)["predicted_cases__avg"] or 0
```

```
maximum_cases = predictions.aggregate(
    Max("predicted_cases")
)["predicted_cases__max"] or 0
```

```
top_disease = predictions.values(
    "disease__disease_name"
).annotate(
    total=Count("id")
).order_by("-total").first()
```

```
highest_region = predictions.values(
    "region__region_name"
```

```

).annotate(
    total=Count("id")
).order_by("-total").first()

recent_predictions = predictions.order_by(
    "-prediction_date"
)[:10]

# =====
# Trend Graph
# =====

dates = predictions.order_by(
    "prediction_date"
).values_list(
    "prediction_date",
    flat=True
)

cases = predictions.order_by(
    "prediction_date"
).values_list(
    "predicted_cases",
    flat=True
)

plt.figure(figsize=(10,5))

plt.plot(
    dates,

```

```

        cases,
        marker="o"
    )

plt.xticks(rotation=45)

plt.title("Prediction Trend")

plt.xlabel("Prediction Date")

plt.ylabel("Predicted Cases")

plt.tight_layout()


graph_folder = os.path.join(
    settings.BASE_DIR,
    "HelathApp",
    "static",
    "graphs"
)

os.makedirs(
    graph_folder,
    exist_ok=True
)

plt.savefig(
    os.path.join(
        graph_folder,
        "dashboard_chart.png"
    )
)

```

```

plt.close()

# =====
# Pie Chart
# =====

plt.figure(figsize=(6,6))

plt.pie(
    [high_count, medium_count, low_count],
    labels=[
        "High",
        "Medium",
        "Low"
    ],
    autopct="%1.1f%%"
)

plt.title(
    "Risk Distribution"
)

plt.savefig(
    os.path.join(
        graph_folder,
        "pie_chart.png"
    )
)

plt.close()

```

```
context = {  
  
    "total_predictions": total_predictions,  
  
    "high_count": high_count,  
  
    "medium_count": medium_count,  
  
    "low_count": low_count,  
  
    "average_cases": round(  
        average_cases,  
        2  
    ),  
  
    "maximum_cases": round(  
        maximum_cases,  
        2  
    ),  
  
    "top_disease": top_disease,  
  
    "highest_region": highest_region,  
  
    "recent_predictions": recent_predictions,  
  
    "trend_graph": "graphs/dashboard_chart.png",  
  
    "pie_chart": "graphs/pie_chart.png",  
  
}
```

```

        return render(
            request,
            "prediction_dashboard.html",
            context
        )

from .models import HealthAlert

def user_health_alerts(request):

    alerts = HealthAlert.objects.filter(
        status="Active"
    ).order_by(
        "-created_at"
    )

    return render(
        request,
        "user_health_alerts.html",
        {
            "alerts": alerts
        }
    )

from .models import Disease

def disease_awareness(request):

    diseases = Disease.objects.all().order_by("disease_name")

    return render(
        request,
        "disease_awareness.html",

```

```

        {
            "diseases": diseases
        }
    )

def disease_details(request, id):

    disease = Disease.objects.get(id=id)

    return render(
        request,
        "disease_details.html",
        {
            "disease": disease
        }
    )

from .models import Hospital

def hospital_map(request):

    hospitals = Hospital.objects.filter(
        status="Approved"
    )

    return render(
        request,
        "hospital_map.html",
        {
            "hospitals": hospitals
        }
    )

```

```

from django.shortcuts import render
from .models import reg, VaccinationCampaign, Region

def user_campaigns(request):

    # Logged user
    user_id = request.session.get("user_id")

    user = reg.objects.get(id=user_id)

    # Dropdown search
    search = request.GET.get("region")

    # User district priority campaigns
    my_campaigns = VaccinationCampaign.objects.filter(
        region__region_name=user.district
    ).exclude(
        status="Cancelled"
    ).order_by(
        "camp_date"
    )

    # Other campaigns
    other_campaigns = VaccinationCampaign.objects.exclude(
        region__region_name=user.district
    ).exclude(
        status="Cancelled"

```

```

).order_by(
    "camp_date"
)

# Region filter
if search:

    my_campaigns = my_campaigns.filter(
        region_id=search
    )

    other_campaigns = other_campaigns.filter(
        region_id=search
    )

regions = Region.objects.all()

return render(
    request,
    "user_campaigns.html",
    {
        "my_campaigns": my_campaigns,
        "other_campaigns": other_campaigns,
        "regions": regions,
        "selected": search,
        "user": user,
    }
)

```

Html codes

```
{% load static %}

<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1">

<title>User Registration</title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.2/css/all.min.css">

<style>

body{

background:linear-gradient(135deg,#4facfe,#00f2fe);

min-height:100vh;

font-family:Arial,Helvetica,sans-serif;

padding:30px;

}

.card{
```

```
max-width:900px;

margin:auto;

border:none;

border-radius:20px;

box-shadow:0px 15px 35px rgba(0,0,0,.25);

overflow:hidden;

}

.card-header{

background:#0d6efd;

color:white;

padding:20px;

text-align:center;

}

.card-body{

padding:35px;

}
```

```
.form-label{  
  
font-weight:bold;  
  
}  
  
.form-control,  
  
.form-select{  
  
border-radius:10px;  
  
height:48px;  
  
}  
  
textarea.form-control{  
  
height:100px;  
  
}  
  
.error{  
  
color:red;  
  
font-size:13px;  
  
}  
  
.preview{  
  
width:130px;
```

```
height:130px;

border-radius:50%;

object-fit:cover;

border:3px solid #ddd;

display:none;

margin:auto;

margin-bottom:15px;

}
```

```
.password-box {

position:relative;

}
```

```
.password-box i {

position:absolute;

right:15px;

top:15px;

cursor:pointer;
```

```
}

.btn-register{

background:#0d6efd;

color:white;

font-size:18px;

padding:12px;

border-radius:10px;

}

.btn-register:hover{

background:#0b5ed7;

}

.location-btn{

margin-top:10px;

}

@media(max-width:768px){

.card-body{

padding:20px;
```

```
}
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="card">
```

```
<div class="card-header">
```

```
<h2>Hospital Registration</h2>
```

```
</div>
```

```
<div class="card-body">
```

```
{% if messages %}
```

```
{% for message in messages %}
```

```
<div class="alert alert-success alert-dismissible fade show">
```

```
{{ message }}
```

```
<button class="btn-close" data-bs-dismiss="alert"></button>
```

```
</div>
```

```
{% endfor %}

{% endif %}

<form method="POST"

enctype="multipart/form-data"

onsubmit="return validateForm()">

{% csrf_token %}

<div class="text-center">

<img id="preview" class="preview">

</div>

<div class="row">

<div class="col-md-6 mb-3">

<label class="form-label">

Name

</label>

<input

type="text"

class="form-control"
```

id="name"

name="name"

required>

</div>

<div class="col-md-6 mb-3">

<label class="form-label">

Age

</label>

<input

type="number"

class="form-control"

id="age"

name="age"

required>

</div>

<div class="col-md-6 mb-3">

<label class="form-label">

Gender

</label>

<select

class="form-select"

name="gender"

required>

<option value="">

Select Gender

</option>

<option value="m">

Male

</option>

<option value="f">

Female

</option>

<option value="o">

Other

</option>

</select>

</div>

<div class="col-md-6 mb-3">

<label class="form-label">

Email

</label>

<input

type="email"

class="form-control"

id="email"

name="email"

required>

```
<span id="emailError" class="error"></span>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Phone

```
</label>
```

```
<input
```

```
type="text"
```

```
class="form-control"
```

```
id="phone"
```

```
name="phone"
```

```
maxlength="10"
```

```
required>
```

```
<span id="phoneError" class="error"></span>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Password

</label>

<div class="password-box">

<input

type="password"

class="form-control"

id="password"

name="password"

required>

<i

class="fa-solid fa-eye"

onclick="togglePassword()">

</i>

</div>

</div>

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Profile Image

```
</label>
```

```
<input
```

```
type="file"
```

```
class="form-control"
```

```
id="image"
```

```
name="image"
```

```
accept="image/*"
```

```
onchange="previewImage(event)">
```

```
<span id="imageError" class="error"></span>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Date of Birth

```
</label>
```

```
<input  
  
type="date"  
  
class="form-control"  
  
name="dob"  
  
required>  
  
</div>  
  
<div class="col-12 mb-3">  
  
<label class="form-label">  
  
Address  
  
</label>  
  
<textarea  
  
class="form-control"  
  
name="address"  
  
required>  
  
</textarea>  
  
</div>  
<div class="row">
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

District

```
</label>
```

```
<select
```

```
class="form-select"
```

```
name="district"
```

```
required>
```

```
<option value="">
```

Select District

```
</option>
```

```
<option value="Thiruvananthapuram">Thiruvananthapuram</option>
```

```
<option value="Kollam">Kollam</option>
```

```
<option value="Pathanamthitta">Pathanamthitta</option>
```

```
<option value="Alappuzha">Alappuzha</option>
```

```
<option value="Kottayam">Kottayam</option>
```

```
<option value="Idukki">Idukki</option>
```

```
<option value="Ernakulam">Ernakulam</option>
```

```
<option value="Thrissur">Thrissur</option>
```

```
<option value="Palakkad">Palakkad</option>
```

```
<option value="Malappuram">Malappuram</option>
```

```
<option value="Kozhikode">Kozhikode</option>
```

```
<option value="Wayanad">Wayanad</option>
```

```
<option value="Kannur">Kannur</option>
```

```
<option value="Kasaragod">Kasaragod</option>
```

```
</select>
```

</div>

<div class="col-md-6 mb-3">

<label class="form-label">

State

</label>

<select

class="form-select"

name="state"

required>

<option value="">

Select State

</option>

<option value="Kerala">

Kerala

</option>

<option value="Tamil Nadu">

Tamil Nadu

</option>

<option value="Karnataka">

Karnataka

</option>

</select>

</div>

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

```
Pincode
```

```
</label>
```

```
<input
```

```
type="text"
```

```
class="form-control"
```

```
id="pincode"
```

```
name="pincode"
```

```
maxlength="6"
```

```
required>
```

```
<span
```

```
class="error"
```

```
id="pinError">
```

```
</span>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

```
Blood Group
```

```
</label>
```

```
<select
```

```
class="form-select"
```

```
name="blood_group"
```

```
required>
```

```
<option value="">
```

Select Blood Group

</option>

<option>A+</option>

<option>A-</option>

<option>B+</option>

<option>B-</option>

<option>AB+</option>

<option>AB-</option>

<option>O+</option>

<option>O-</option>

</select>

</div>

<div class="col-md-6 mb-3">

<label class="form-label">

Emergency Contact

</label>

<input

type="text"

class="form-control"

id="emergency"

name="emergency_contact"

maxlength="10"

required>

<span

class="error"

id="emergencyError">

```
</span>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Latitude

```
</label>
```

```
<input
```

```
type="number"
```

```
step="any"
```

```
class="form-control"
```

```
id="latitude"
```

```
name="latitude"
```

```
>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label class="form-label">
```

Longitude

```
</label>
```

```
<input
```

```
type="number"
```

```
step="any"
```

```
class="form-control"
```

```
id="longitude"
```

```
name="longitude"
```

```
>
```

```
</div>
```

```
<div class="col-md-6 mb-3 d-grid">
```

```
{% comment %} <button  
type="button"  
class="btn btn-success location-btn"  
onclick="getLocation()">
```

```
<i class="fa-solid fa-location-crosshairs"></i>
```

Get Current Location

```
</button> {% endcomment %}
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<div  
id="locationStatus"  
style="margin-top: 12px; color: green; font-weight: bold;">  
</div>
```

```
</div>
```

```
</div>
```

```
<hr>
```

```
<button  
type="submit"
```

```
class="btn btn-register w-100">
```

```
<i class="fa-solid fa-user-plus"></i>
```

Register

```
</button>
```

```
<div class="text-center mt-4">
```

Already have an account?

```
<a href="{% url 'login' %}">
```

Login

```
</a>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
<script>
```

```
// ----- Image Preview -----
```

```
function previewImage(event){
```

```
    const preview=document.getElementById("preview");
```

```
    preview.src=URL.createObjectURL(event.target.files[0]);
```

```

        preview.style.display="block";

    }

    // ----- Password Show / Hide -----

    function togglePassword(){

        let pass=document.getElementById("password");

        let icon=document.querySelector(".password-box i");

        if(pass.type==="password"){

            pass.type="text";

            icon.classList.remove("fa-eye");

            icon.classList.add("fa-eye-slash");

        }

        else{

            pass.type="password";

            icon.classList.remove("fa-eye-slash");

            icon.classList.add("fa-eye");

        }

    }

```

```

}

// ----- Browser Location -----

function getLocation(){

    if(navigator.geolocation){

        document.getElementById("locationStatus").innerHTML="Getting
location...";

        navigator.geolocation.getCurrentPosition(showPosition,errorPosition);

    }

    else{

        alert("Geolocation not supported.");

    }

}

function showPosition(position){

    document.getElementById("latitude").value=position.coords.latitude;

    document.getElementById("longitude").value=position.coords.longitude;

    document.getElementById("locationStatus").innerHTML="Location captured

```

```

successfully.";

}

function errorPosition(){

    document.getElementById("locationStatus").style.color="red";

    document.getElementById("locationStatus").innerHTML="Unable to fetch
location.";

}

// ----- Form Validation -----

function validateForm(){

    let valid=true;

    document.querySelectorAll(".error").forEach(function(e){

        e.innerHTML="";

    });

    // Name

    let name=document.getElementById("name").value.trim();

```

```

if(!/^[A-Za-z ]{3,50}$/.test(name)){

    document.getElementById("nameError").innerHTML="Enter valid name";

    valid=false;

}


// Age

let age=parseInt(document.getElementById("age").value);

if(age<18 || age>100){

    document.getElementById("ageError").innerHTML="Age must be between
18 and 100";

    valid=false;

}


// Email

let email=document.getElementById("email").value;

let emailPattern=/^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if(!emailPattern.test(email)){

    document.getElementById("emailError").innerHTML="Invalid email";

```

```

        valid=false;

    }

    // Phone

    let phone=document.getElementById("phone").value;

    if(!/^[6-9][0-9]{9}$/.test(phone)){

        document.getElementById("phoneError").innerHTML="Enter valid mobile
number";

        valid=false;

    }

    // Password

    let password=document.getElementById("password").value;

    let passPattern=/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).{8,}$/;

    if(!passPattern.test(password)){

        document.getElementById("passError").innerHTML="Minimum 8
characters, uppercase, lowercase, number & special character";

        valid=false;

    }

```

```
// Pincode

let pin=document.getElementById("pincode").value;

if(! /^[1-9][0-9]{5}$/.test(pin)){

    document.getElementById("pinError").innerHTML="Invalid pincode";

    valid=false;

}


// Emergency Contact

let emergency=document.getElementById("emergency").value;

if(! /^[6-9][0-9]{9}$/.test(emergency)){

    document.getElementById("emergencyError").innerHTML="Invalid
emergency contact";

    valid=false;

}


// Image Validation

let image=document.getElementById("image").files[0];
```

```

if(image){

    let allowed=["image/jpeg","image/png","image/jpg"];

    if(!allowed.includes(image.type)){

        document.getElementById("imageError").innerHTML="Only JPG and
PNG images allowed";

        valid=false;

    }

    if(image.size>2*1024*1024){

        document.getElementById("imageError").innerHTML="Image size
should be below 2 MB";

        valid=false;

    }

}

// Location

let lat=document.getElementById("latitude").value;

let lon=document.getElementById("longitude").value;

if(lat=="" || lon==""){

```

```

        alert("Please click 'Get Current Location' before registering.");

        valid=false;

    }

    return valid;

}

</script>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc
ript>

</body>

</html>

<table class="table table-bordered">

<tr>
    <th>Day</th>
    <th>Predicted Cases</th>
</tr>

{% for p in prediction %}

<tr>

<td>{{ forloop.counter }}</td>

```

```
<td>{{ p|floatformat:2 }}</td>
```

```
</tr>
```

```
{% endfor %}
```

```
</table>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Submit Disease Report</title>
```

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
```

```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.5.2/css/all.min.css">
```

```
<style>
```

```
body{
background:#eef3f8;
}
```

```
.card{
border:none;
border-radius:15px;
```

```
}

.card-header{
background:#0B3D91;
color:white;
font-size:24px;
font-weight:bold;
}

label{
font-weight:bold;
}

.form-control,
.form-select{
height:45px;
}

textarea{
height:120px !important;
}

</style>

</head>

<body>

<div class="container mt-5">

<div class="row justify-content-center">

<div class="col-lg-9">
```

```
<div class="card shadow">
```

```
<div class="card-header">
```

```
<i class="fa-solid fa-notes-medical"></i>
```

Submit Disease Case Report

```
</div>
```

```
<div class="card-body">
```

```
{% if messages %}
```

```
    {% for message in messages %}
```

```
        <script>
```

```
            alert("{{ message }}");
```

```
        </script>
```

```
    {% endfor %}
```

```
{% endif %}
```

```
<form method="POST" onsubmit="return validateForm();">
```

```
{% csrf_token %}
```

```
<div class="row">
```

```
<div class="col-md-6 mb-3">
```

```
<label>Disease</label>
```

```
<select name="disease" class="form-select" required>
```

```
<option value="">Select Disease</option>
```

```

{% for d in diseases %}

<option value="{{ d.id }}">

{{ d.disease_name }}

</option>

{% endfor %}

</select>

</div>

<div class="col-md-6 mb-3">

<label>Region</label>

<select name="region" class="form-select" required>

<option value="">Select Region</option>

{% for r in regions %}

<option value="{{ r.id }}">

{{ r.region_name }}

</option>

{% endfor %}

```

```
</select>
```

```
</div>
```

```
<div class="col-md-3 mb-3">
```

```
<label>Total Cases</label>
```

```
<input
```

```
type="number"
```

```
name="case_count"
```

```
class="form-control"
```

```
id="case_count"
```

```
min="1"
```

```
required>
```

```
</div>
```

```
<div class="col-md-3 mb-3">
```

```
<label>Male Cases</label>
```

```
<input
```

```
type="number"
```

```
name="male_cases"
```

```
class="form-control"
```

```
id="male_cases"
```

```
min="0"
```

```
>
```

```
</div>
```

```
<div class="col-md-3 mb-3">
```

```
<label>Female Cases</label>
```

```
<input  
type="number"  
name="female_cases"  
id="female_cases"  
class="form-control"  
min="0"  
>
```

```
</div>
```

```
<div class="col-md-3 mb-3">
```

```
<label>Children Cases</label>
```

```
<input  
type="number"  
name="children_cases"  
class="form-control"  
id="children_cases"  
min="0"  
>
```

```
</div>
```

```
<div class="col-md-6 mb-3">
```

```
<label>Report Date</label>
```

```
<input type="date"  
name="report_date"
```

```
id="report_date"
max="{% now 'Y-m-d' %}"
required>
```

```
</div>
```

```
<div class="col-md-12 mb-3">
```

```
<label>Remarks</label>
```

```
<textarea
name="remarks"
class="form-control"></textarea>
```

```
</div>
```

```
<div class="text-center">
```

```
<button
class="btn btn-success">
```

```
<i class="fa-solid fa-floppy-disk"></i>
```

Submit Report

```
</button>
```

```
<a
href="{% url 'hospital_home' %}"
```

```
class="btn btn-secondary">
```

Back

```
</a>
</div>
</div>
</form>
</div>
</div>
</div>
</div>
</div>
</body>
<script>
function validateForm() {

    let total = parseInt(document.getElementById("case_count").value) || 0;
    let male = parseInt(document.getElementById("male_cases").value) || 0;
    let female = parseInt(document.getElementById("female_cases").value) || 0;
    let children = parseInt(document.getElementById("children_cases").value) || 0;

    if (total !== (male + female + children)) {
        alert("Total cases must equal Male + Female + Children cases.");
        return false;
    }

    return true;
}
</script>
</html>
```

Standardization of the Coding

5.1 Programming Standards and Coding Practices

To ensure software quality, maintainability, security, and reliability, various programming standards and coding practices were followed during the development of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. These practices improved code readability, minimized errors, and simplified future system maintenance and enhancements.

5.1.1 Standardization of Coding

Coding standards were followed throughout the development process to maintain consistency and improve code quality. Meaningful variable names, proper indentation, modular programming techniques, and reusable functions were adopted to enhance readability and maintainability.

The Django Framework's recommended coding conventions were followed while developing models, views, templates, forms, APIs, and database operations. Proper comments and documentation were added wherever necessary to simplify understanding of the source code.

The use of standardized coding practices enabled easier debugging, testing, maintenance, and future enhancement of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system.

5.1.2 Code Efficiency

Code efficiency was considered throughout the implementation of the system to ensure optimal resource utilization and improved performance. Efficient database queries were used to minimize execution time and reduce unnecessary processing.

Django's Object Relational Mapping (ORM) was utilized for efficient database operations. Reusable functions, modular programming, and optimized logic were

adopted to reduce code duplication and improve maintainability.

The application was optimized to provide faster response times for operations such as user authentication, hospital management, disease reporting, healthcare resource management, AI-based disease forecasting, healthcare access mapping, vaccination campaign management, health alert generation, and administrative monitoring.

5.1.3 Error Handling

Proper error handling mechanisms were implemented to ensure reliable system operation and prevent unexpected failures. The system validates user inputs before processing and displays appropriate error messages whenever invalid data is entered.

Exception handling techniques were used to manage runtime errors, database exceptions, authentication failures, and invalid user requests.

Common errors such as invalid login credentials, duplicate registrations, incomplete disease reports, invalid healthcare resource entries, missing mandatory fields, and unauthorized access attempts are handled gracefully, improving user experience and maintaining system stability.

5.1.4 Parameter Passing

Parameter passing techniques were used to transfer information between different components of the application. Parameters are passed between views, templates, models, and functions to facilitate communication among system modules.

URL parameters are used to identify specific records for viewing hospital details, updating disease reports, managing vaccination campaigns, generating health alerts, and monitoring healthcare resources. Function parameters are used to process user inputs and perform business operations efficiently.

Proper parameter handling ensures accurate execution of system operations and minimizes incorrect data processing.

5.1.5 Validation Checks

Validation checks were implemented at both the client side and server side to ensure data accuracy and integrity. These validation mechanisms prevent invalid information from entering the system and improve overall data quality.

The following validation checks are implemented:

- Verification of mandatory fields.
- Email format validation.
- Mobile number validation.
- Password confirmation and strength validation.
- Hospital registration validation.
- Disease report validation.
- Healthcare resource validation.
- Duplicate username and email detection.
- Numeric field validation.
- Date and time format validation.

Django Forms and Model Validators were extensively used to perform these validation checks, ensuring that only valid and consistent healthcare information is stored in the database.

5.1.6 Advantages of Coding Standards

The adoption of coding standards and programming practices provided several benefits:

- Improved code readability and maintainability.
- Reduced software defects and programming errors.
- Enhanced application performance.
- Simplified debugging and testing.
- Improved data accuracy through validation mechanisms.
- Increased system security, reliability, and scalability.
- Facilitated future system modifications and enhancements.

The implementation of these programming standards significantly contributed to the overall quality, security, maintainability, and performance of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system

Testing

6.1 Introduction

System testing is one of the most important phases in the Software Development Life Cycle (SDLC). The primary objective of testing is to identify defects, verify system functionality, and ensure that the developed software satisfies user requirements. Testing improves software quality by detecting errors before deployment and ensuring that all system components function correctly.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system underwent various testing procedures to verify its correctness, reliability, security, and performance. Each module was tested individually and collectively to ensure smooth interaction among system components. The testing process helped identify defects, validate functionality, and improve the overall quality of the application.

Proper testing ensures that the system operates according to specifications and provides a reliable platform for disease monitoring, healthcare access mapping, healthcare resource management, AI-based disease forecasting, and public health decision-making.

6.2 Objectives of Testing

The primary objective of software testing is to verify that the developed application performs according to user requirements and design specifications. Testing ensures that all functionalities operate correctly and produce the expected outputs for valid inputs.

Another objective is to identify and eliminate software defects before deployment. Testing also evaluates system performance, security, usability, reliability, and AI prediction accuracy under different operating conditions. Through systematic testing, the quality of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system was significantly improved.

6.3 Testing Strategy

A comprehensive testing strategy was adopted during the development of the system. Different testing techniques were applied at various stages to ensure complete verification of system functionalities.

Testing activities included Unit Testing, Integration Testing, System Testing, Validation Testing, User Acceptance Testing, Performance Testing, and Security Testing. Each testing phase focused on specific aspects of the system and ensured that both functional and non-functional requirements were satisfied before deployment.

6.4 Unit Testing

Unit testing was performed to verify the functionality of individual modules independently.

Modules tested include: User Registration , Hospital Registration , Disease Reporting, Healthcare Resource Management , AI-Based Disease Forecasting , Healthcare Access Mapping , Vaccination Campaign Management , Health Alert Generation , User Authentication

Unit testing helped identify coding errors, validation issues, and logical inconsistencies during the early stages of development.

6.5 Integration Testing

Integration testing verified communication between interconnected modules.

The system modules including User Management, Hospital Management, Disease Reporting, Healthcare Resource Management, AI Disease Forecasting, Healthcare Access Mapping, Vaccination Management, and Administrative Functions were tested together to ensure smooth data flow.

Integration testing confirmed that all modules communicated correctly through the centralized database.

6.6 System Testing

System testing evaluated the complete application as an integrated system.

Functions such as user registration, login, hospital management, disease reporting, healthcare resource monitoring, vaccination campaign management, disease forecasting, healthcare access mapping, health alert generation, and administrative monitoring were thoroughly tested.

The results confirmed that the system satisfied all functional and operational requirements.

6.7 Validation Testing

Validation testing ensured that the application satisfied user requirements and business objectives.

Different scenarios were executed to verify that administrators, hospitals, field workers, and users could successfully perform their assigned tasks. Disease reporting, AI prediction, healthcare access analysis, vaccination management, and administrative operations were validated successfully.

6.8 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with sample users including healthcare administrators, hospital staff, and field workers.

Participants tested hospital registration, disease reporting, healthcare resource updates, vaccination campaign management, AI disease forecasting, healthcare access mapping, and health alert generation.

Feedback indicated that the system was easy to use, responsive, and met user expectations. Minor usability improvements identified during testing were

incorporated into the final version.

6.9 Performance Testing

Performance testing evaluated the responsiveness, stability, and efficiency of the application.

Operations such as login, disease reporting, AI prediction, healthcare resource updates, healthcare mapping, and vaccination management were executed repeatedly.

The results showed fast response time, efficient database operations, and stable performance under normal system load.

6.10 Security Testing

Security testing verified that the application protects sensitive healthcare information and prevents unauthorized access.

Testing focused on: User Authentication, Password Validation, Session Management, Role-Based Access Control and Data Protection

The results confirmed that only authorized users could access their permitted modules and that healthcare data remained secure.

6.11 Advantages of Testing

Testing provided several benefits during system development:

- Early identification and correction of defects.
- Improved reliability and stability.
- Enhanced system security.
- Improved application performance.
- Accurate AI-based disease forecasting.
- Efficient healthcare data management.

- Increased user satisfaction.
- Reduced maintenance effort.
- Ensured compliance with project requirements.

Comprehensive testing significantly improved the quality, reliability, usability, and performance of the system.

6.12 Summary

Various testing techniques were successfully carried out on the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system to evaluate its functionality, performance, security, and usability. These included Unit Testing, Integration Testing, System Testing, Validation Testing, User Acceptance Testing, Performance Testing, and Security Testing.

The testing results confirmed that all modules operated correctly and interacted seamlessly. The application demonstrated reliable performance, accurate AI-based disease forecasting, secure authentication, efficient healthcare data management, and an intuitive user interface.

Overall, the testing process verified that the system satisfied both functional and non-functional requirements. The successful completion of all testing activities confirmed the quality, stability, security, and reliability of the application, making it ready for deployment and practical use.

System Security Mechanisms

7.1 Introduction

Security is one of the most important aspects of any web-based application. Since the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system stores sensitive healthcare information such as user details, hospital records, disease reports, healthcare resources, vaccination data, health alerts, and AI-generated disease forecasts, appropriate security measures are essential to protect the system from unauthorized access and misuse. The application incorporates various security mechanisms including authentication, authorization, password protection, session management, database security, and input validation to ensure secure and reliable operation.

7.2 Security Objectives

The primary objective of system security is to protect healthcare information from unauthorized access and cyber threats. The system ensures that only authenticated users can access application resources and perform authorized operations. It also aims to maintain data confidentiality, preserve data integrity, and ensure continuous availability of healthcare services for legitimate users.

7.3 Authentication Mechanism

Authentication is used to verify the identity of users before granting access to the system. The application utilizes Django's authentication framework to validate user credentials. Users must enter a valid username and password to log in. Based on their assigned role, authenticated users can access the corresponding system modules, while invalid login attempts are denied.

7.4 Authorization and Access Control

The system implements Role-Based Access Control (RBAC) to manage user permissions. Different access rights are assigned to Administrators, Hospitals, Field Workers, and Users. Administrators manage the entire platform, hospitals update disease reports and healthcare resources, field workers conduct vaccination campaigns and regional surveys, and users can access healthcare information and alerts. This

ensures users can access only the functionalities permitted for their roles.

7.5 Password Security

Passwords are securely stored using Django's password hashing mechanism. Passwords are never stored in plain text, making unauthorized access to user accounts significantly more difficult even if the database is compromised. This enhances overall account security and protects sensitive healthcare information.

7.6 Session Management

The application uses Django session management to maintain secure user login sessions. Sessions remain active until the user logs out or the session expires after a period of inactivity. Proper session management helps prevent unauthorized access and improves application security.

7.7 Database Security

The database stores important information related to users, hospitals, disease reports, healthcare resources, vaccination campaigns, health alerts, AI-generated disease forecasts, and administrative activities. Database access is protected through authentication, authorization, and application-level security controls. Regular database backups are maintained to prevent data loss and support system recovery.

7.8 Input Validation

Input validation is performed on all user forms and data entry fields to ensure data accuracy and system security. Validation checks include email verification, password validation, mandatory field validation, numeric field validation, and data type verification. These measures prevent invalid data entry and reduce security vulnerabilities.

7.9 Data Confidentiality

Data confidentiality ensures that sensitive healthcare information is accessible only to authorized users. The system stores user information, hospital records, disease reports,

healthcare resources, vaccination records, and AI prediction results that must be protected from unauthorized access.

Role-based access control restricts access to confidential information according to user responsibilities. Maintaining confidentiality enhances user trust and protects sensitive healthcare data.

7.10 Data Integrity

Data integrity ensures the accuracy, consistency, and reliability of healthcare information throughout its lifecycle. The system utilizes database constraints, validation rules, and controlled access mechanisms to prevent unauthorized modifications and maintain data consistency.

Primary keys, foreign keys, and validation procedures ensure that healthcare records remain accurate and reliable during system operation.

7.11 Backup and Recovery Mechanism

Data loss may occur due to hardware failures, software errors, accidental deletion, or cyber incidents. Therefore, regular database backups are maintained to protect critical healthcare information.

The recovery mechanism enables administrators to restore previous database states whenever necessary, minimizing downtime and ensuring continuous system availability.

7.12 Security Threats and Prevention

Web applications are vulnerable to various security threats including unauthorized access, SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), phishing attacks, and data breaches.

To reduce these risks, the system implements secure authentication, password hashing, role-based access control, input validation, CSRF protection, session management, and continuous monitoring. These mechanisms help maintain system security and protect

sensitive healthcare information.

7.13 Advantages of Security Mechanisms

The implementation of security mechanisms provides several benefits:

- Protects sensitive healthcare information.
- Prevents unauthorized access.
- Ensures secure user authentication.
- Maintains data confidentiality and integrity.
- Protects against common cyber attacks.
- Supports reliable backup and recovery.
- Improves user trust and system reliability.
- Enhances overall application security.

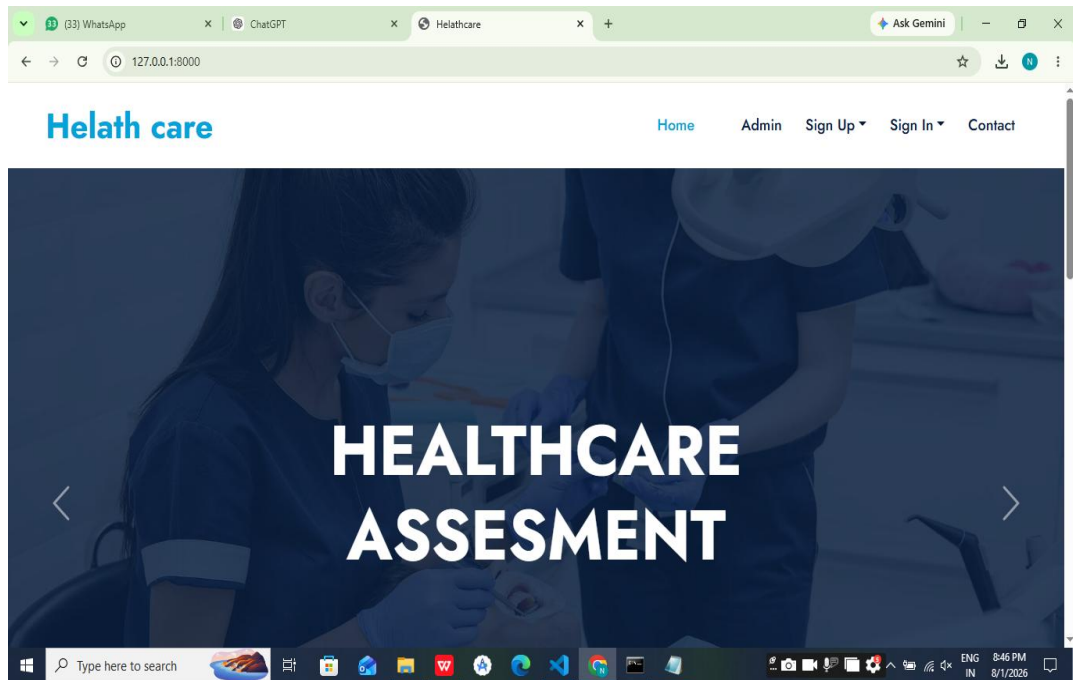
These security measures ensure a secure and dependable healthcare management platform.

7.14 Summary

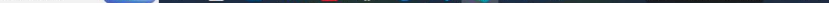
Security mechanisms play a vital role in ensuring the safe and reliable operation of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. Various techniques such as authentication, authorization, password hashing, input validation, session management, database security, and backup procedures have been successfully implemented.

These mechanisms ensure the confidentiality, integrity, and availability of healthcare data while protecting the system against potential security threats. Overall, the implemented security framework enhances system reliability and provides a secure environment for Administrators, Hospitals, Field Workers, Healthcare Authorities, and Users.

Results



The screenshot displays a web form titled 'Assign Mobile Healthcare Unit'. The form is set against a light blue background and contains several input fields and buttons. The fields are arranged in two columns: 'Vehicle Number' (text input with 'KL-08-AB-1234') and 'Driver Name' (text input); 'Assign Field Worker' (dropdown menu with 'Select Worker') and 'Assign Region' (dropdown menu with 'Select Region'); 'Assigned Date' (text input with 'mm/dd/yyyy' and a calendar icon) and 'Status' (text input with 'Assigned'). Below these is a 'Remarks' section with a text area labeled 'Enter remarks if any...'. At the bottom of the form are two buttons: a green 'Assign' button and a grey 'Back' button. The browser's address bar shows '127.0.0.1:8000/add_mobile_unit/'.



(33) WhatsApp
ChatGPT
Disease List
Ask Gemini

127.0.0.1:8000/disease_list/

Disease List
+ Add Disease

Back to Dashboard

ID	Disease Name	Disease Type	Severity	Description	Action
1	Dengue	Viral	High	Dengue is a mosquito-borne viral disease spread by Aedes mosquitoes. Common symptoms include high fever, headache, muscle pain, and skin rash. Early diagnosis and treatment help prevent severe complications.	Edit Delete
2	COVID-19	Viral	High	Respiratory viral infection caused by SARS-CoV-2.	Edit Delete
3	Malaria	Parasitic	High	Mosquito-borne disease caused by Plasmodium parasites.	Edit Delete
4	Typhoid	Bacterial	Medium	Bacterial infection caused by Salmonella Typhi.	Edit Delete
5	Nipah Virus	Viral	High	Rare but highly dangerous viral infection.	Edit Delete
6	Chickenpox	Viral	Low	Contagious disease causing itchy skin rash.	Edit Delete
7	Cholera	Bacterial	High	Water-borne disease causing severe diarrhea.	Edit Delete

(33) WhatsApp
ChatGPT
Doctor List
Ask Gemini

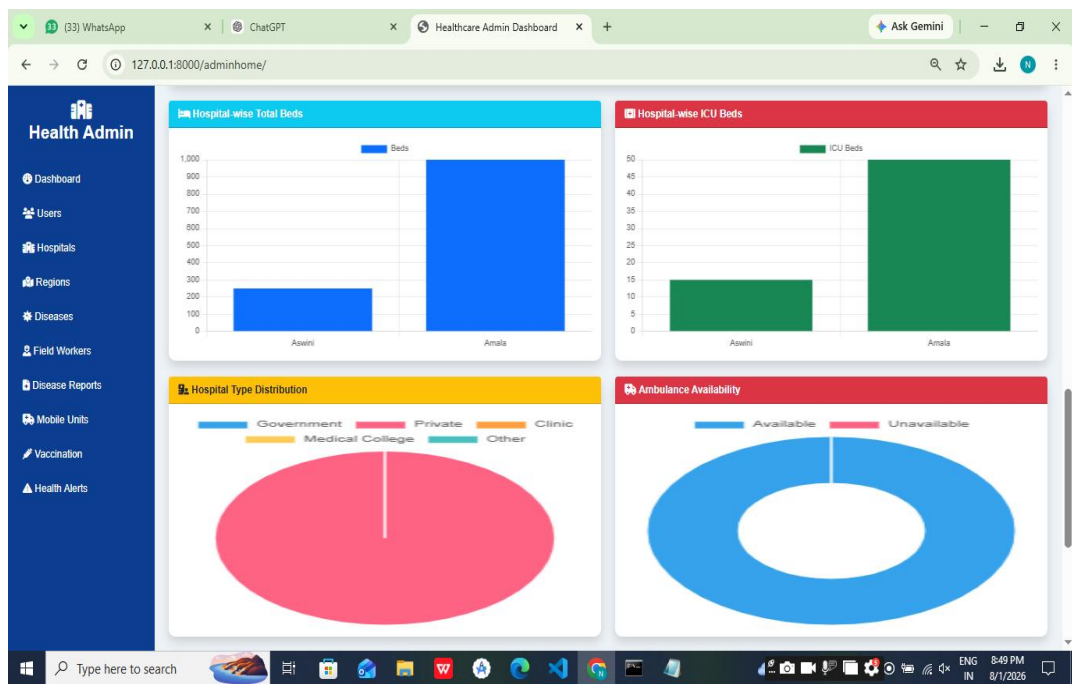
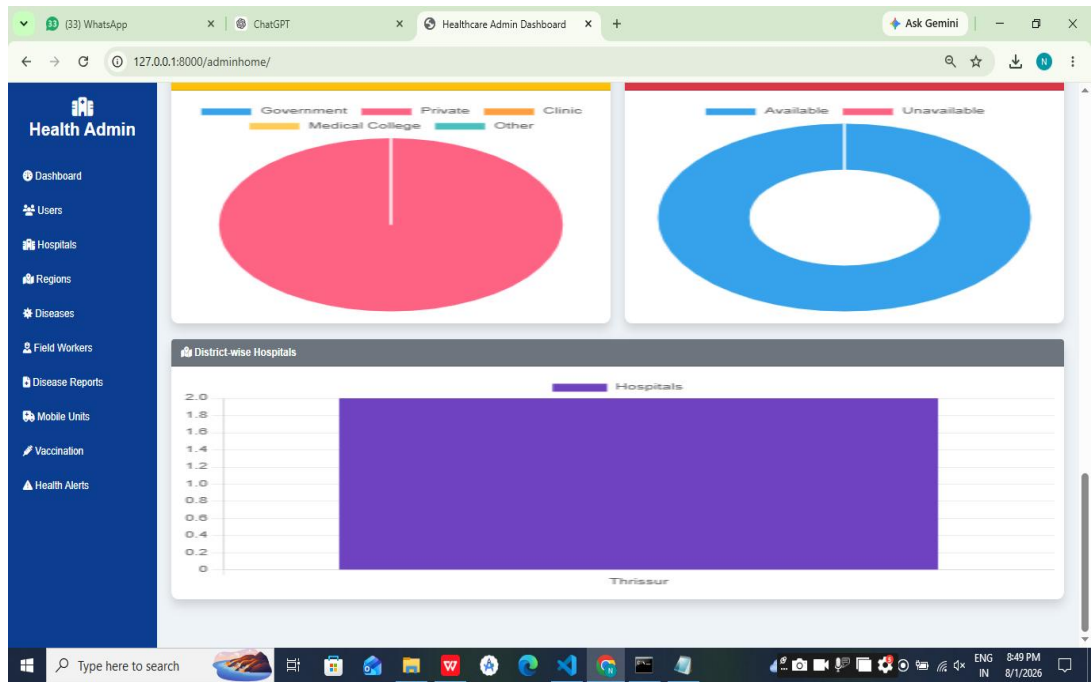
127.0.0.1:8000/doctor_list/

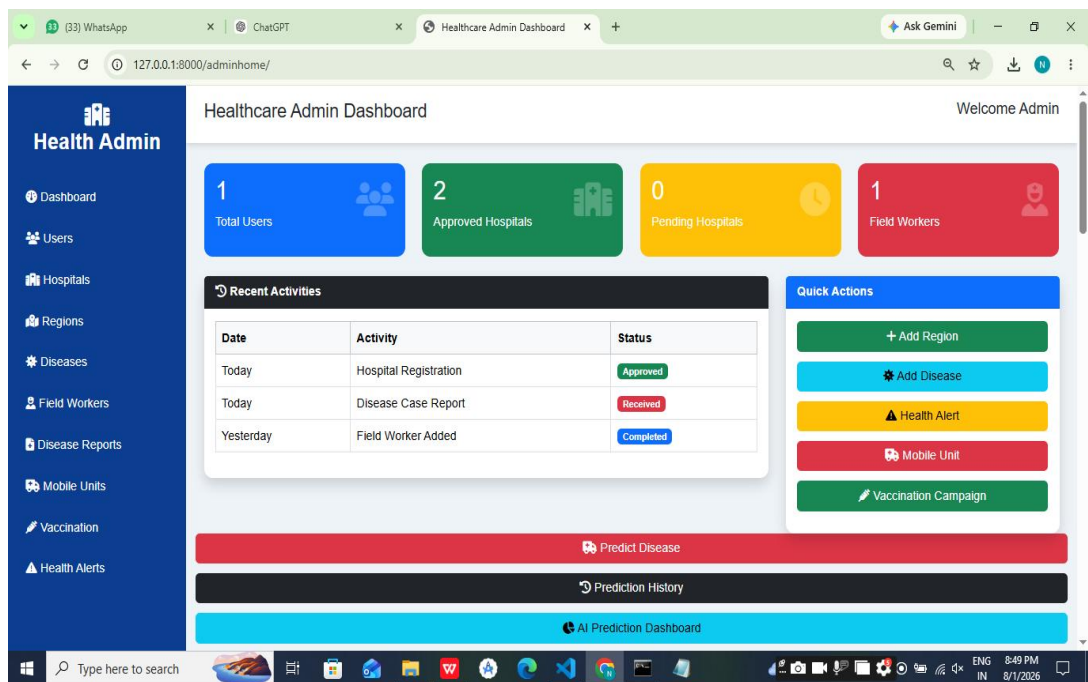
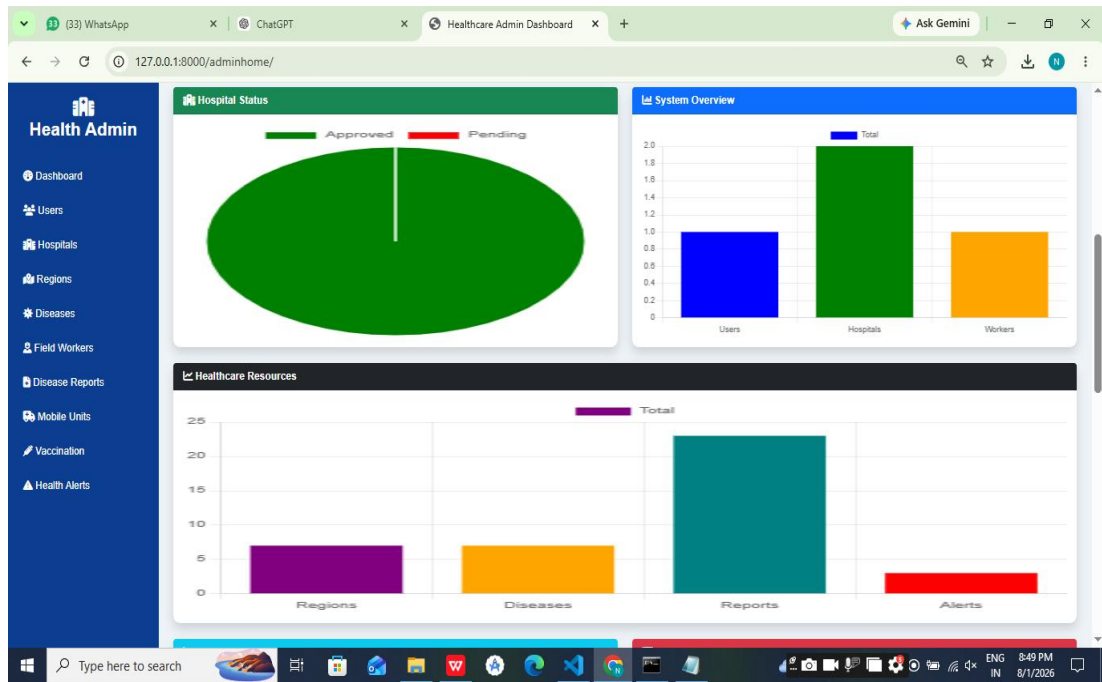
Doctor List

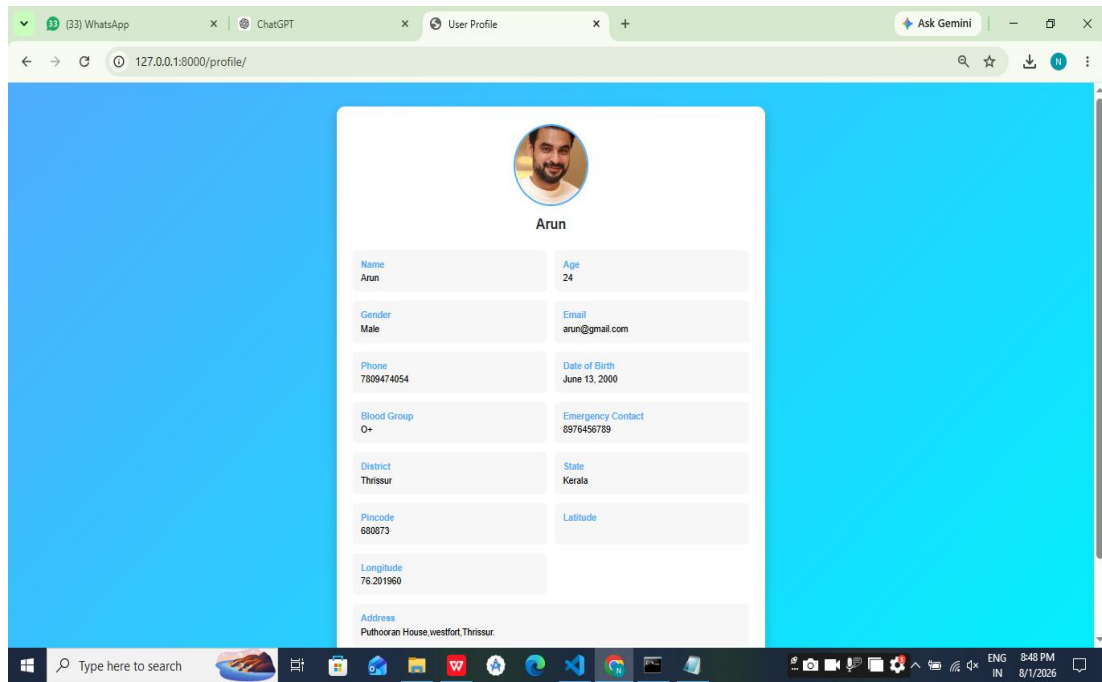
Name	Qualification	Specialization	Hospital	Email	Phone	Address	Admin Name	Admin Phone	Status	Action
Aswini	Private	KHAS4354	342423	Aswini@gmail.com	7809474032	Aswini Hospital,Aswini Junction,Thrissur,Kerala	Sreejith	9876564543	Approved	Already Approved
Amala	Private	KL4354655	JHjg53566	amala@gmail.com	90784587634	Amala Hospital Amalanagar po, Thrissur	amal	7876734673	Approved	Already Approved

Type here to search

ENG IN 8:50 PM 8/1/2026



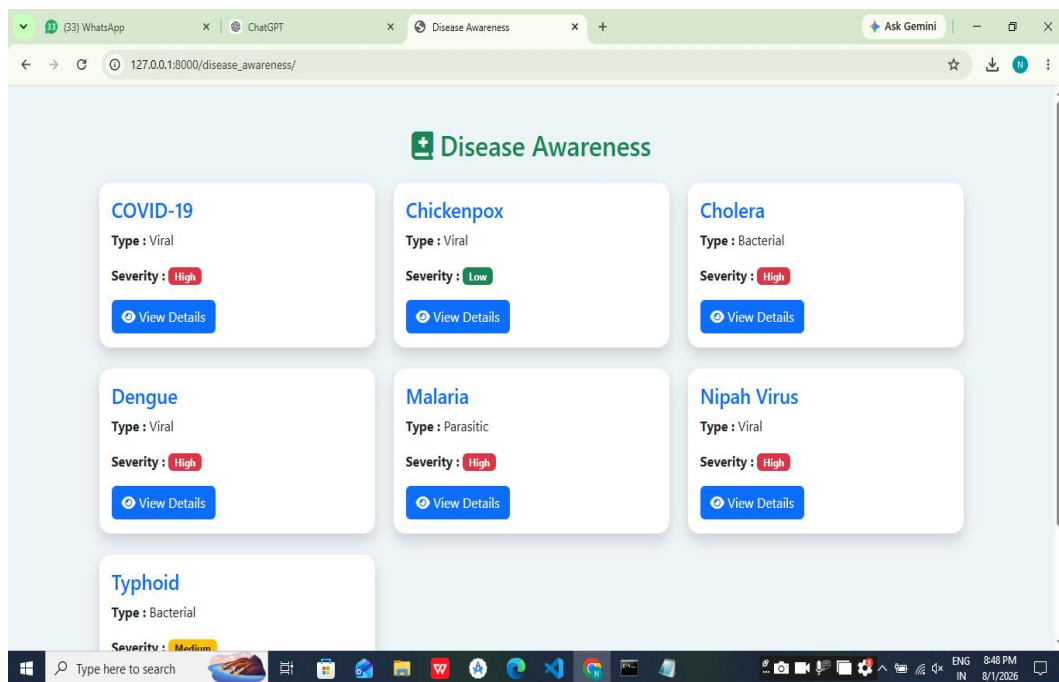
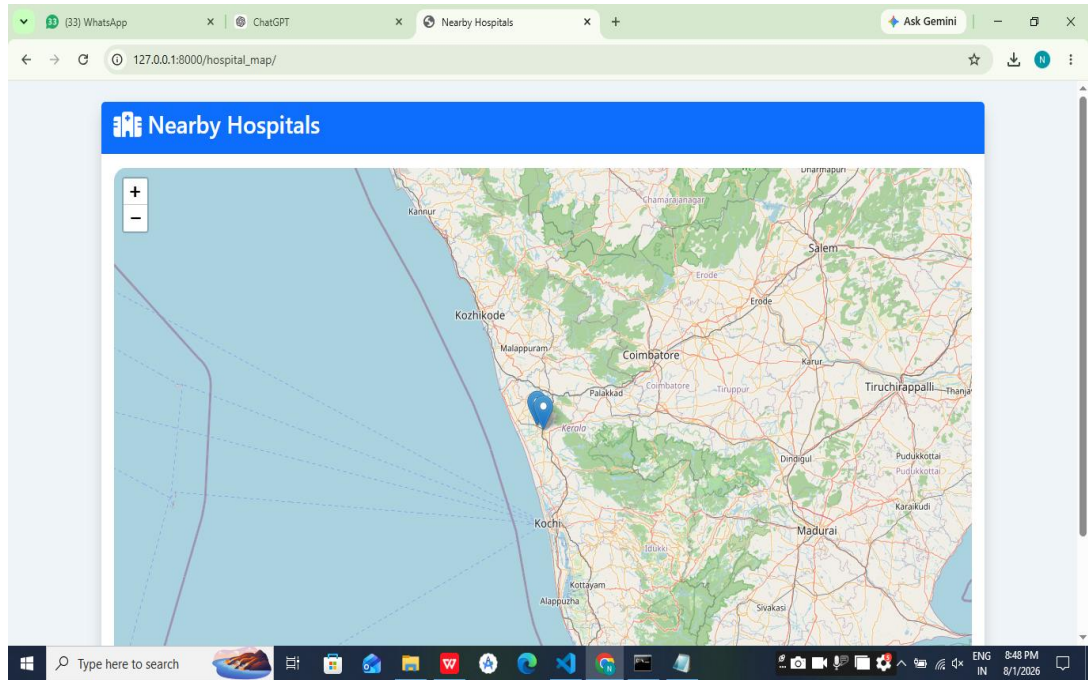


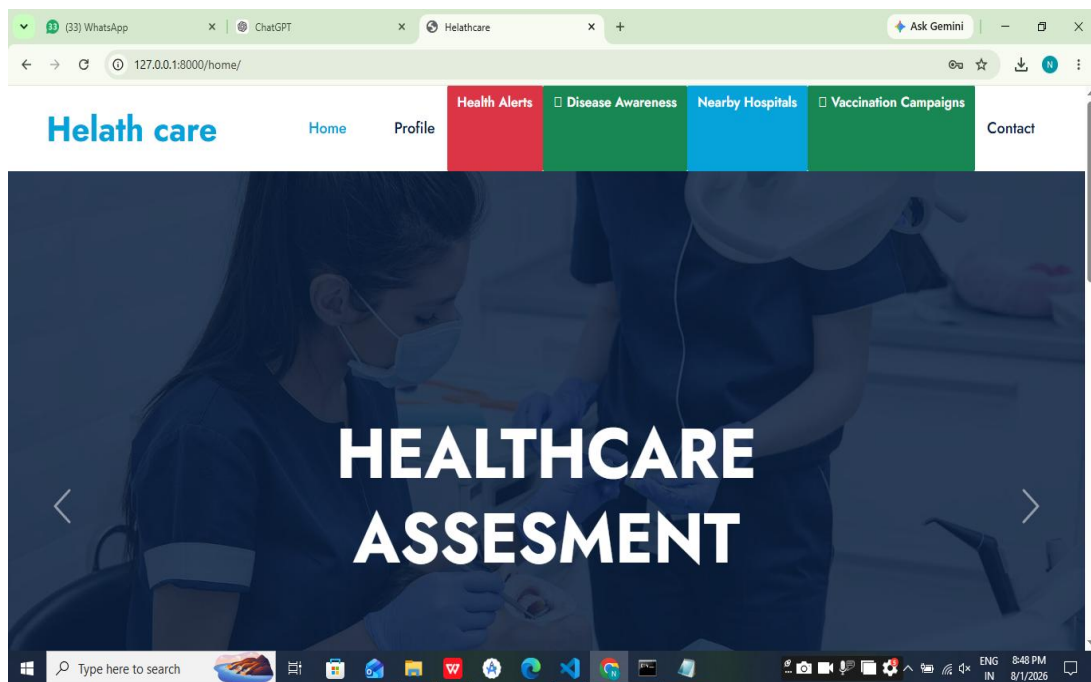
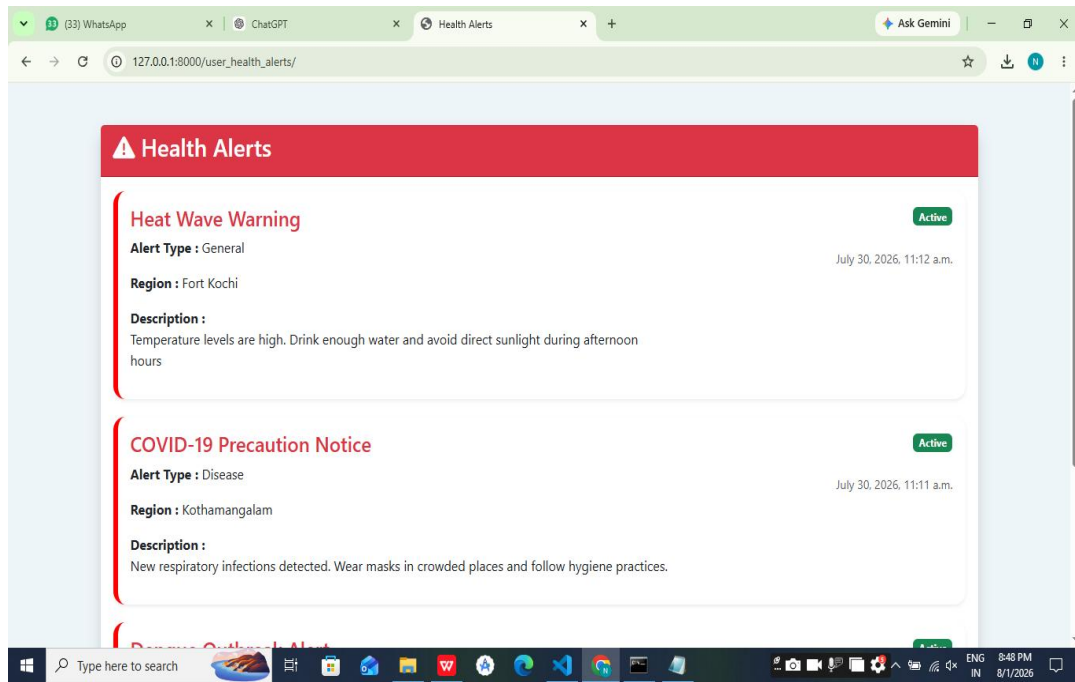


Vaccination Campaigns

Campaign Name	Disease	Region	Date	Status
Dengue Prevention Camp	Dengue	Ernakulam City	Aug. 7, 2026	Completed
COVID-19 Vaccination Drive	COVID-19	Ernakulam City	Aug. 18, 2026	Completed







Browser tabs: (33) WhatsApp, ChatGPT, Login, Ask Gemini

Address bar: 127.0.0.1:8000/login/

Login

Email

Password

Don't have an account? [Registr](#)

Windows taskbar: Type here to search, 8:47 PM, 8/1/2026

Browser tabs: (33) WhatsApp, ChatGPT, User Registration, Ask Gemini

Address bar: 127.0.0.1:8000/registration/

User Registration

Name 	Age
Gender Select Gender	Email saron
Phone 	Password ***
Profile image Choose File No file chosen	Date of Birth mm/dd/yyyy
Address 	
District Select District	State Select State
Pincode 	Blood Group Select Blood Group
Emergency Contact 	Latitude
Longitude 	

Already have an account? [Login](#)

Windows taskbar: Type here to search, 8:47 PM, 8/1/2026

Future Enhancements

Although the current AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system provides an intelligent platform for disease monitoring, healthcare accessibility analysis, and AI-based disease prediction, several enhancements can further improve its accuracy, efficiency, and usability.

Advanced AI and Deep Learning Models

Future versions of the system can incorporate advanced Artificial Intelligence techniques such as Deep Learning, Ensemble Learning, and Time-Series Forecasting models to improve the accuracy of disease outbreak prediction. These models can analyze complex healthcare patterns and provide more reliable disease forecasts.

Real-Time IoT Healthcare Integration

The system can be integrated with Internet of Things (IoT) devices and smart healthcare equipment to collect real-time patient health data, hospital resource availability, and environmental conditions. This integration will enable faster disease detection and more accurate healthcare monitoring.

Mobile Application Support

Dedicated Android and iOS mobile applications can be developed to allow healthcare professionals, field workers, and users to access the system anytime and anywhere. Mobile applications can provide instant disease alerts, healthcare notifications, vaccination updates, and healthcare facility information.

GIS and Satellite Data Integration

Future enhancements can include advanced Geographic Information System (GIS) mapping and satellite imagery for accurate visualization of disease hotspots and healthcare accessibility. This will assist healthcare authorities in identifying underserved regions and planning healthcare resource distribution more effectively.

Multilingual Support

The application can be enhanced with support for multiple regional and international languages. This feature will improve accessibility for users from different linguistic backgrounds and encourage wider adoption of the system.

Wearable Health Device Integration

The system can be integrated with wearable health devices such as smartwatches and fitness trackers to collect real-time health parameters, enabling early detection of disease symptoms and continuous patient health monitoring.

Cloud-Based Big Data Analytics

Future versions may utilize cloud computing and big data analytics to process large-scale healthcare datasets efficiently. This will improve disease forecasting accuracy, support real-time analytics, and enhance healthcare decision-making at regional and national levels.

Emergency Response and Ambulance Tracking

An emergency response module with GPS-based ambulance tracking can be integrated into the system. This feature will help users locate nearby emergency medical services and enable faster response during disease outbreaks and medical emergencies.

Summary

These future enhancements can transform the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system into a comprehensive smart healthcare platform. By incorporating advanced AI techniques, IoT integration, GIS technology, mobile accessibility, wearable device support, cloud analytics, and emergency response services, the system will provide more accurate disease forecasting, improve healthcare accessibility, optimize resource allocation, and support efficient public health management.

Conclusion

10.1 Introduction

The conclusion represents the final stage of the project report and provides a comprehensive summary of the work carried out during the development of the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system. It highlights the major achievements of the project, evaluates the extent to which the project objectives have been successfully accomplished, and discusses the overall impact, effectiveness, and usefulness of the developed system in addressing real-world healthcare challenges. The conclusion also reflects upon the development process, the benefits offered by the system, and its potential for future growth and enhancement.

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system was developed to address challenges in disease surveillance, healthcare accessibility, and public health management. Traditional healthcare monitoring methods often lack real-time analysis, centralized data management, and predictive capabilities, making it difficult for healthcare authorities to respond quickly to disease outbreaks and allocate healthcare resources efficiently.

To overcome these challenges, the proposed system introduces a centralized, AI-powered, web-based platform that integrates healthcare data, disease reports, healthcare facility information, vaccination campaigns, and AI-based disease forecasting to support informed public health decisions.

The system was developed using modern web technologies including Python, Django Framework, SQLite Database, HTML, CSS, JavaScript, and Bootstrap, providing a secure, scalable, and user-friendly platform for healthcare management.

Overall, the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system provides an effective solution for improving disease surveillance, healthcare accessibility, resource management, and public health decision-making.

10.2 Project Summary

The project began with a detailed study of existing healthcare monitoring systems and the challenges associated with disease surveillance, healthcare accessibility, and resource allocation. It was observed that the absence of a centralized AI-based healthcare management platform limited the ability of healthcare authorities to monitor disease trends and respond effectively to emerging health issues.

Based on this analysis, a web-based solution was designed to assist healthcare administrators, hospitals, field workers, and users in managing healthcare information and predicting disease trends. The application was developed using Python and the Django Framework, with SQLite as the database management system. The user interface was developed using HTML, CSS, JavaScript, and Bootstrap to provide a responsive and user-friendly experience.

The application integrates modules such as User Management, Hospital Management, Disease Reporting, Healthcare Access Mapping, AI-Based Disease Forecasting, Vaccination Campaign Management, Mobile Healthcare Unit Management, Health Alert Generation, and Administrative Control into a single healthcare platform.

10.3 Achievement of Objectives

The primary objective of the project was to develop a secure, intelligent, and centralized healthcare management platform capable of monitoring disease outbreaks, forecasting disease trends, and improving healthcare accessibility. This objective has been successfully achieved through the implementation of all required modules and functionalities.

The system enables healthcare organizations to manage disease reports, monitor healthcare resources, analyze disease trends using Artificial Intelligence, generate health alerts, manage vaccination campaigns, and improve healthcare accessibility. Administrators can efficiently monitor healthcare activities and maintain the overall system.

By implementing these features, the project successfully achieved its objective of improving disease surveillance, healthcare planning, and public health decision-making.

10.4 Benefits of the System

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system provides several benefits to healthcare authorities, hospitals, field workers, and users.

The system enables healthcare organizations to monitor disease outbreaks, predict disease trends, manage healthcare resources, and improve healthcare accessibility through a centralized platform. AI-based forecasting assists healthcare authorities in making proactive decisions, while health alerts and vaccination management improve public health preparedness.

Administrators benefit from centralized monitoring and management of healthcare information, enabling efficient planning and resource allocation.

10.5 Evaluation of the System

The developed system was thoroughly evaluated using various testing techniques including Unit Testing, Integration Testing, System Testing, Performance Testing, Security Testing, and User Acceptance Testing.

The testing results confirmed that the application performs reliably under different operating conditions and processes healthcare information efficiently. The system demonstrated satisfactory performance in terms of response time, AI prediction accuracy, security, usability, and reliability.

Users who participated in testing found the application easy to use and effective for monitoring healthcare services and disease trends. The evaluation confirmed that the system meets the specified requirements and is suitable for practical healthcare applications.

10.6 Challenges Faced During Development

Several challenges were encountered during the development of the project.

One of the major challenges was designing a database capable of efficiently managing healthcare information such as disease reports, healthcare facilities, vaccination records, healthcare resources, and AI prediction results while maintaining data consistency.

Another challenge involved integrating Artificial Intelligence with the web application to generate accurate disease forecasts and developing an intuitive user interface suitable for different categories of users. These challenges were successfully addressed through careful planning, modular development, continuous testing, and effective utilization of the Django framework.

10.7 Contribution of the Project

The AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system contributes significantly to modern healthcare management by providing a centralized and AI-driven platform for disease surveillance and healthcare accessibility analysis.

The project demonstrates how Artificial Intelligence and modern web technologies can be utilized to solve real-world public healthcare challenges. By integrating disease reporting, healthcare access mapping, AI forecasting, vaccination management, and health alert generation, the system provides a comprehensive healthcare management solution.

One of the major contributions of the project is the promotion of data-driven healthcare decision-making. Healthcare authorities can utilize AI-generated insights to identify disease hotspots, allocate healthcare resources efficiently, and implement preventive measures before disease outbreaks become severe.

The modular architecture also provides a strong foundation for future enhancements and technological integration.

10.8 Future Scope

Although the current system effectively fulfills its primary objectives, there is considerable scope for future enhancements.

Future versions can incorporate advanced deep learning models to improve disease prediction accuracy. Integration with IoT-enabled medical devices and wearable health sensors can facilitate real-time patient monitoring. Mobile applications can improve accessibility for healthcare professionals and users.

Additional enhancements may include telemedicine integration, multilingual support, electronic health record (EHR) integration, cloud-based analytics, GIS-enabled disease hotspot visualization, emergency ambulance tracking, and smart healthcare resource optimization.

These improvements will further enhance the capabilities of the system and support intelligent public healthcare management.

10.9 Final Conclusion

The **AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting** system successfully provides a comprehensive, intelligent, and efficient solution for monitoring disease outbreaks, forecasting disease trends, and improving healthcare accessibility. The application was developed with the objective of supporting healthcare authorities in making timely and informed public health decisions.

The system integrates several important functionalities including User Management, Hospital Management, Disease Reporting, Healthcare Access Mapping, AI-Based Disease Forecasting, Vaccination Campaign Management, Mobile Healthcare Unit Management, Health Alert Generation, and Administrative Monitoring within a centralized platform. These integrated modules work together to improve disease surveillance, healthcare planning, and resource management.

The project successfully achieved its intended objectives by delivering a secure, scalable, and user-friendly web application capable of enhancing healthcare accessibility and public health management. Healthcare authorities benefit from accurate disease forecasts, timely health alerts, improved resource allocation, and efficient monitoring of healthcare services.

The implementation using Python, Django Framework, SQLite Database, HTML, CSS, JavaScript, and Bootstrap contributed significantly to the successful development of the application. Through systematic planning, efficient database management, AI

integration, and comprehensive testing, the system demonstrated stable and reliable performance.

In conclusion, the AI for Real-Time Healthcare Access Mapping and Disease Trend Forecasting system represents a valuable contribution to smart healthcare. With future enhancements such as advanced AI models, IoT integration, cloud analytics, telemedicine support, GIS visualization, and wearable health device integration, the platform has the potential to evolve into a comprehensive intelligent healthcare decision-support system that significantly improves disease surveillance, healthcare accessibility, emergency response, and overall public health management.

Bibliography

- [1] Django Software Foundation, *Django Documentation*,
Available at: <https://docs.djangoproject.com/>
- [2] Python Software Foundation, *Python Documentation*,
Available at: <https://docs.python.org/>
- [3] SQLite Consortium, *SQLite Documentation*,
Available at: <https://www.sqlite.org/docs.html>
- [4] Bootstrap Team, *Bootstrap Documentation*,
Available at: <https://getbootstrap.com/docs/>
- [5] Mozilla Developer Network, *JavaScript Guide*,
Available at: <https://developer.mozilla.org/>
- [6] W3C, *HTML5 Specification*, Available at: <https://www.w3.org/TR/html5/>
- [7] W3C, *CSS Documentation*, Available at: <https://www.w3.org/Style/CSS/>

- [8] Ian Sommerville, *Software Engineering*, 10th Edition, Pearson Education, 2016.
- [9] Roger S. Pressman, *Software Engineering: A Practitioner's Approach*, 8th Edition, McGraw-Hill Education, 2015.
- [10] Elmasri, R. and Navathe, S., *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2017.